Active Learning with a Poisson–Gaussian-Process Model:
Variational Inference, Information Utility, and the Diverging-Utility Problem

Mathematical reference (domain-neutral)

July 11, 2026

Abstract

This is a self-contained mathematical reference for a sequential active-learning problem built on a Poisson–Gaussian-process model. A latent function λ over an input space (inputs are real-valued signals on a bounded two-dimensional grid) is modeled by a Gaussian process with an arc-cosine kernel over a structured prior covariance C; observations are counts with a Poisson likelihood whose rate is an exponential link of the latent; the model is fit by sparse variational inference; and active learning chooses the next input to maximize expected information gain about λ. The document develops, in one uniform notation: the generative model and the arc-cosine kernel with its structured prior $C(\mathrm{Amp}, \beta, \rho, \boldsymbol{\xi}_0)$ (Part I); sparse variational inference, the eigenspace representation of the posterior, and coordinate-ascent training (Part II); the information utility for active learning together with the predictive-conditioning, subspace, divergence, and numerical results beneath it (Part III); and guided diffusion, which extends the loop from selecting inputs out of a pool to synthesizing them by steering a diffusion model with the utility gradient (Part IV). The central object is the *diverging-utility problem*: the arc-cosine kernel is positively homogeneous in the input, so the acquisition utility grows without bound as the input scale grows and drives the optimizer to the boundary of the bounded input box. The divergence is proved and the attempted kernel fixes (a normalized arc-cosine kernel, a saturating arc-sin kernel) are analyzed; finding a kernel that keeps the utility bounded on the bounded domain while retaining good fit is left as an open problem.

Contents

Scope, conventions, and the open problem

This is a self-contained mathematical reference for a sequential *active-learning* problem built on a Poisson–Gaussian-process model. A latent function $\lambda(x)$ over an input space is modeled by a Gaussian process (GP) with an arc-cosine kernel over a structured prior covariance C. Each input $x \in \mathbb{R}^d$ is a real-valued signal on a two-dimensional grid of d grid points (e.g. a 108×108 grid, $d = 11664$) confined to a bounded box. Observations are nonnegative integer counts with a Poisson likelihood whose rate is an exponential link of the latent, $f = \exp(A\lambda + \lambda_0)$. At each round the active-learning rule chooses the next input so as to maximize the expected information gain about λ. Part 9 extends the loop from *selecting* the next input out of a fixed pool to *synthesizing* it with a guided diffusion model.

The open problem (the purpose of this document). The arc-cosine kernel is positively homogeneous in the input: its self-value $K(x, x) = x^\mathsf{T} C x + \sigma_0^2$ grows with $\|x\|$. Under utility maximization the posterior variance therefore grows like the *square* of the input scale, the acquisition utility diverges, and — because inputs are confined to a bounded box — the optimizer drives them to the boundary of the box (saturation). Two remedies are presented and analyzed: a normalized arc-cosine kernel and a saturating arc-sin kernel (§8.5). Neither is satisfactory in practice: the normalized kernel is scale-invariant but fits poorly. It is an **open problem** to find a kernel that simultaneously (i) keeps the acquisition utility bounded on the bounded input domain and (ii) retains good predictive fit. The divergence is proved in Part 8 (§8.4); the attempted fixes are §8.5.

Conventions.

- **One symbol, one concept** throughout; the full table is Appendix A. A few symbols that collide across the literature are disambiguated there (the kernel smoothness ρ vs. the posterior correlation ρ_λ; the prior width β vs. the diffusion noise schedule β_t; the three entropies $H_{\text{marg}}, H_{\text{noise}}, H_{\text{cond}}$).
- **Counts.** The count observation is written y; where the count index appears (as in the marginal entropy sum and its truncation $r_{\max}$) it is written r, denoting the same quantity, $y \equiv r$.
- **Scope of this rendering.** Content that is purely implementation-specific (automatic-vs-analytic differentiation machinery, the vector–Jacobian-product derivation, the low-level representation and online-update bookkeeping, and run/status details) has been omitted; the mathematics of the model, inference, utility, and the divergence problem is complete.

How to read this document. The four parts are logically ordered. Part 2–3 sets up the generative model and the arc-cosine kernel with its structured prior. Part II (4–6) develops sparse variational inference, the eigenspace representation of the posterior, and the coordinate-ascent training of the model parameters. Part III (7–8) is the active-learning core: the information utility and the proofs, subspace theory, and numerical analysis beneath it — including the divergence result and the attempted kernel fixes that define the open problem. Part IV (9) is guided diffusion for input synthesis. Cross-references connect the utility gradient in Parts III and IV back to the kernel input-gradient of Part I, and the training back to the ELBO of Part II.

Part I
Foundations: the generative model and its prior

1 Introduction: the sequential active-learning problem

1.1 Setup

We study a sequential active-learning problem for a Poisson–Gaussian-process model of a latent function $\lambda(x)$ over inputs x. An input $x \in \mathbb{R}^d$ is a real-valued signal on a 2-D grid of d grid points (e.g. a 108×108 grid, so $d = 108^2 = 11664$); grid point i sits at grid coordinate $\boldsymbol{\xi}_i$, and the input domain is a bounded box (e.g. $[0,1]^d$). For a single input x the model produces a scalar latent value $\lambda(x)$, an exponential-link rate $f(x) = \exp(A\,\lambda(x) + \lambda_0)$, and a Poisson count observation $y \sim \text{Poisson}(f(x))$ (§2). None of the formulas below assume the particular dimension $d = 11664$.

The latent function λ is unknown a priori and must be inferred from the counts. We place a Gaussian-process (GP) prior on λ and update its posterior online as counts arrive. The setting is *sequential*: after each observation the model is refit and used to choose the next input, so the inputs presented depend on the observations already collected.

1.2 The active-learning goal

The inputs are not presented in a fixed order. At each round the next input is chosen to be maximally informative about λ — this is *active learning*. A candidate input x^* is scored by an information *utility* $U_{\text{std}}(x^*)$ (Part III) measuring the expected reduction in uncertainty about λ if x^* were observed next; the input maximizing the utility over the available pool is chosen. A second variant U_{DA} scores information about the rate over the input (data) distribution $p(x)$. Because scoring and selecting from a fixed pool is limited by the pool, a final stage (Part IV) instead *synthesizes* an input that maximizes the utility, using a diffusion prior over inputs steered by the utility gradient. All three stages consume two derivatives of the GP prior kernel that are established in this part: the input gradient $\nabla_x K$ (10) (utility and diffusion guidance) and the hyperparameter gradients $\partial C/\partial\theta$ (12) (model fitting).

1.3 Roadmap of the document

The document proceeds in the order the pipeline runs.

- **Part I (this part) — the model and its prior.** The generative model (§2): a latent GP over input space, an exponential link, and Poisson counts. The prior kernel (§3): the order-1 arc-cosine (ReLU) kernel built on a structured spatial prior covariance $C(\text{Amp}, \beta, \rho, \boldsymbol{\xi}_0)$ that encodes a local, smooth weighting centred on the localized region, together with the two kernel gradients the rest of the pipeline needs.
- **Part II — inference.** The Poisson likelihood breaks conjugacy, so the posterior over the latent is approximated by a sparse variational GP on M inducing points with Gaussian posterior $q(\widetilde{\lambda}) = \mathcal{N}(m, V)$, optimized against the evidence lower bound (ELBO); the model is trained by an EM-style coordinate-ascent loop (E-, F-, M-steps).

- **Part III — active learning.** The information utility: the marginal count entropy H_{marg}, the production utility $U_{\text{std}} = H_{\text{marg}} - \mathbb{E}[H_{\text{noise}}]$, and the distribution-aware research variant $U_{\text{DA}} = H_{\text{marg}} - \mathbb{E}_{x\sim p(x)}[H_{\text{cond}}]$; a deep layer of conditioning, subspace, and divergence results grounded in the kernel of this part.
- **Part IV — synthesis.** Guided diffusion: a denoising diffusion prior whose reverse process is steered by $\nabla_x U_{\text{std}}$ (which flows through $\nabla_x K$) to synthesize high-utility inputs.

The central open problem. A single thread runs from the prior of this part to the analysis of Part III. The arc-cosine kernel is positively homogeneous in the input: its self-value $K(x,x) = x^{\mathsf{T}}Cx + \sigma_0^2$ (§3.2) grows with the input norm. Under utility maximization the posterior variance then grows with the square of the input scale, so the utility *diverges* and the optimizer drives inputs to the boundary of the bounded box (saturation). Whether a kernel exists that both (i) keeps the utility bounded on the bounded input domain and (ii) fits well is the central *open problem*; it is set up here and analyzed in Part III (with two candidate fixes, a normalized arc-cosine kernel and a saturating arc-sin kernel).

2 The generative model

2.1 Latent Gaussian process, exponential link, Poisson counts

The count observation y_i (also written r_i, or n for a novel test input) for input x_i is a Poisson draw whose rate is an exponential link applied to a latent Gaussian-process function λ.

Definition 2.1 (Generative model)**.** With a zero-mean GP prior over input space,

$$\lambda(\cdot) \;\sim\; \mathcal{GP}(0,\; K)\,, \qquad f(x) \;=\; \exp\big(A\,\lambda(x) + \lambda_0\big), \qquad y_i \;\sim\; \text{Poisson}\big(f(x_i)\big). \tag{1}$$

Here $\lambda(x) \in \mathbb{R}$ is the *latent function* (what the GP learns; *not* the rate); $f(x) > 0$ is the *rate* (the Poisson rate parameter); $A > 0$ is the *gain*, scaling how strongly the latent modulates the rate; and $\lambda_0 \in \mathbb{R}$ is the *bias* / log-baseline rate, so that $f = e^{\lambda_0}$ when $\lambda = 0$. The log-rate is $g(x) = A\,\lambda(x) + \lambda_0$, giving $f = e^g$.

For numerical stability the gain is constrained positive and bounded, $A \in (0, 10]$, and the bias to $\lambda_0 \in [-50, 50]$.

The GP prior is defined by the property that any finite set of latent values $\{\lambda(x_1), \ldots, \lambda(x_n)\}$ is jointly Gaussian, $\lambda(X) \sim \mathcal{N}(0, K)$ with $[K]_{ij} = K(x_i, x_j) = \text{Cov}[\lambda(x_i), \lambda(x_j)]$. The mean is zero. The kernel K is fixed in *form* before seeing data but its entries depend on the inputs through evaluation; its full definition is §3.

2.2 Why the GP models the latent, not the rate

Modelling λ as Gaussian and pushing it through $f = \exp(A\lambda + \lambda_0)$ means the rate is *log-normally* distributed: we learn a scaled, shifted log of the Poisson rate rather than the rate itself. Two properties make this the right choice for count data.

- *Positivity for free.* The exponential link guarantees $f > 0$ for any real latent value. A Gaussian prior placed *directly* on the rate would assign mass to negative (invalid) rates.

- *Smoothness where it belongs.* The GP prior imposes spatial structure (locality and smoothness, §3) on the unconstrained latent λ, not on the constrained rate. The prior belief — that λ is a spatially local, smooth function of the input — is naturally expressed on the unconstrained log-rate, which is where the structure lives.

Remark 2.2 (Poisson breaks conjugacy)**.** For a Gaussian likelihood the posterior and predictive would be closed-form. With the Poisson likelihood the marginal $p(y \mid X, \theta) = \int \text{Poisson}(y \mid f(x))\, \mathcal{N}(\lambda \mid 0, K_\theta)\, d\lambda$ has no closed form, so neither the exact posterior $p(\lambda \mid y, X, \theta)$ nor the exact predictive can be computed directly. This is what forces the variational treatment of Part II; here we fix only the model and its prior.

3 The arc-cosine kernel and the structured spatial prior

The prior covariance K is a first-order arc-cosine (ReLU) kernel built on a structured input-covariance matrix C. This section owns the definitions of both K and C, their parameterization and constraints, and the two gradients ($\nabla_x K$, $\partial C/\partial\theta$) that the utility, the diffusion guidance, and the M-step consume.

3.1 The order-1 arc-cosine kernel

Definition 3.1 (Order-1 arc-cosine kernel)**.** For two input vectors $x, x' \in \mathbb{R}^d$,

$$K(x,x') \;=\; \frac{1}{\pi}\,\mathcal{M}\,J(\theta), \qquad \mathcal{M} = \sqrt{v_x\, v_{x'}}, \qquad \cos\theta = \frac{x^\mathsf{T} C x' + \sigma_0^2}{\mathcal{M}}, \qquad J(\theta) = \sin\theta + (\pi-\theta)\cos\theta, \tag{2}$$

with self-magnitudes (prior variances) $v_x = x^\mathsf{T} C x + \sigma_0^2$ and $v_{x'} = x'^\mathsf{T} C x' + \sigma_0^2$, cross-term $x^\mathsf{T} C x' + \sigma_0^2$, magnitude $\mathcal{M}$, angle θ, and order-1 angular term J. Equivalently, expanding the angle,

$$\theta = \arccos\left(\frac{x^\mathsf{T} C x' + \sigma_0^2}{\sqrt{x^\mathsf{T} C x + \sigma_0^2}\,\sqrt{x'^\mathsf{T} C x' + \sigma_0^2}} \right). \tag{3}$$

Here σ_0^2 is a constant *bias variance* and $C \in \mathbb{R}^{d\times d}$ is the structured spatial prior covariance (§3.3).

Remark 3.2 (Order and ReLU-network interpretation)**.** This is the order-$n = 1$ member of the Cho–Saul (2009) arc-cosine family ($n = 0$ is Heaviside/step, $n = 1$ is ReLU); the angular term $J(\theta) = \sin\theta + (\pi-\theta)\cos\theta$ is exactly the ReLU case. The kernel is the covariance of the output of an infinite-width, single-hidden-layer ReLU network whose input-to-hidden weights are drawn $w \sim \mathcal{N}(0, C)$, with the bias contributing σ_0^2:

$$K(x,x') \;=\; \mathbb{E}_{w\sim\mathcal{N}(0,C)}\big[\,\text{ReLU}(w^\mathsf{T} x)\,\text{ReLU}(w^\mathsf{T} x')\,\big] \quad \text{(up to the } 1/\pi \text{ constant, with bias } \sigma_0^2\text{)}. \tag{4}$$

Thus C is literally the prior covariance of the network weights across grid points. Shaping C (§3.3) injects the prior belief that λ depends on the input through a spatially *local* and *smooth* region of grid points.

3.2 Non-stationarity and the self-kernel identity

Proposition 3.3 (Self-kernel identity)**.** *The prior variance at any input is the C-quadratic form plus the bias variance:*

$$K(x,x) \;=\; v_x \;=\; x^\mathsf{T} C x + \sigma_0^2. \tag{5}$$

Proof. At $x' = x$ the cross-term equals $v_x = \mathcal{M}$, so $\cos\theta = 1$, $\theta = 0$, and $J(0) = \sin 0 + (\pi - 0)\cos 0 = \pi$. Hence $K(x, x) = \mathcal{M}\, J(0)/\pi = \mathcal{M} = \sqrt{v_x\, v_x} = v_x$. □

Remark 3.4 (Non-stationarity and its consequence for active learning)**.** Equation (5) shows K depends on the actual input values through $x^\mathsf{T} C x$, not only on $x - x'$: the kernel is *non-stationary* and the prior variance v_x grows with the input norm (in the C-metric). This has a direct downstream effect: the active-learning utility is driven by posterior variance, so utility maximization is biased toward high-norm inputs unless the kernel is normalized. For exactly this reason there are normalized and saturating siblings of the base kernel — a normalized arc-cosine kernel ($\bar{K} = J(\theta)/\pi$, constant $\bar{K}(x, x) = 1$) and a saturating arc-sin kernel (bounded diagonal) — analyzed in Part III as fixes to the norm-scaling divergence. This part keeps the focus on the base arc-cosine kernel.

3.3 The structured spatial prior covariance C

C encodes that the localized support of the prior is spatially local and smooth. It is *constant with respect to the input x* — it depends only on the hyperparameters $(\text{Amp}, \beta, \rho, \boldsymbol{\xi}_0)$. Grid point i sits at grid coordinate $\boldsymbol{\xi}_i$ on a normalized $[-1, 1] \times [-1, 1]$ grid, and $\boldsymbol{\xi}_0 = (\varepsilon_{0x}, \varepsilon_{0y})$ is the center of the localized region.

Definition 3.5 (Structured spatial prior)**.** With an amplitude Amp, a per-grid-point locality weight α_i, and a pairwise smoothing factor C_{smooth},

$$C_{ij} \;=\; \text{Amp}\,\alpha_i\,[C_{\text{smooth}}]_{ij}\,\alpha_j \;=\; \text{Amp}\,\exp\!\left(-\frac{\|\boldsymbol{\xi}_i - \boldsymbol{\xi}_0\|^2}{4\beta^2}\right)\exp\!\left(-\frac{\|\boldsymbol{\xi}_i - \boldsymbol{\xi}_j\|^2}{2\rho^2}\right)\exp\!\left(-\frac{\|\boldsymbol{\xi}_j - \boldsymbol{\xi}_0\|^2}{4\beta^2}\right), \tag{6}$$

where

$$\begin{aligned} \alpha_i &= \exp\!\left(-\frac{\|\boldsymbol{\xi}_i - \boldsymbol{\xi}_0\|^2}{4\beta^2}\right) \quad \text{(locality weight)}, \\ [C_{\text{smooth}}]_{ij} &= \exp\!\left(-\frac{\|\boldsymbol{\xi}_i - \boldsymbol{\xi}_j\|^2}{2\rho^2}\right) \quad \text{(smoothness kernel)}, \end{aligned} \tag{7}$$

followed by a symmetrization $C \leftarrow (C + C^\mathsf{T})/2$.

Interpretation of the natural parameters (defaults Amp $= 1.0$, $\beta = \rho = 0.1$ on the $[-1, 1]$ grid):

- Amp — overall *amplitude*: a positive gain (default 1.0) on the structured covariance C, scaling the prior variance of the latent. It multiplies the C-quadratic part of the self-magnitude $v_x = x^\mathsf{T} C x + \sigma_0^2$ (not the bias σ_0^2); because it enters the kernel inside $\mathcal{M} = \sqrt{v_x v_{x'}}$ and $\cos\theta$, it rescales K non-linearly rather than as a plain output scale. Unlike $\beta, \rho, \boldsymbol{\xi}_0$ it sets the magnitude, not the spatial shape, of the localized weighting.
- β — *half-width* of the localized region: the locality weight α is a Gaussian of standard deviation $\sqrt{2}\,\beta$ in grid-distance, reaching e^{-1} at dist $= 2\beta$; for $\beta = 0.1$ the localized region spans roughly ± 0.2 ($\approx 20\%$ of the axis).
- ρ — smoothness *standard deviation* (correlation length) of the RBF-like C_{smooth} (variance ρ^2): grid points closer than ρ are strongly correlated, grid points beyond 3ρ nearly uncorrelated.
- $\boldsymbol{\xi}_0$ — the 2-D center of the localized region on $[-1, 1]^2$.

Together, α localizes weight around $\boldsymbol{\xi}_0$ and C_{smooth} correlates nearby grid points — a localized, smooth weight prior.

3.4 Raw parameterization and constraints

For optimizer stability the model is parameterized by unconstrained log-space *raw* parameters rather than β and ρ directly:

$$\begin{aligned} \text{raw\_m2log2beta} = -2\log(2\beta) \quad &\Rightarrow \quad \exp(\text{raw\_m2log2beta}) = \frac{1}{4\beta^2}, \\ \text{raw\_mlog2rho2} = -\log(2\rho^2) \quad &\Rightarrow \quad \exp(\text{raw\_mlog2rho2}) = \frac{1}{2\rho^2}, \end{aligned} \tag{8}$$

and $\sigma_0 = \exp(\text{raw\_sigma\_0})$ under a positivity constraint (so the bias variance added in (2) is $\sigma_0^2 = (\exp(\text{raw\_sigma\_0}))^2$). The exponentials of the raw parameters are used *directly* as the exponent coefficients, so that $\alpha_i = \exp(-\frac{1}{4\beta^2}\|\boldsymbol{\xi}_i - \boldsymbol{\xi}_0\|^2)$ and $[C_{\text{smooth}}]_{ij} = \exp(-\frac{1}{2\rho^2}\|\boldsymbol{\xi}_i - \boldsymbol{\xi}_j\|^2)$ reproduce exactly the natural-parameter form (6) (numerically, $\beta = \rho = 0.1 \Rightarrow 1/(4\beta^2) = 25$, $1/(2\rho^2) = 50$). The raw parameters invert back to the natural values, $\beta = \frac{1}{2}\exp(-\text{raw\_m2log2beta}/2)$, $\rho = \frac{1}{\sqrt{2}}\exp(-\text{raw\_mlog2rho2}/2)$, and round-trip cleanly.

Param	Constraint mechanism	Bounds (natural)	Bounds (raw)
σ_0	$\sigma_0 = \exp(\text{raw\_sigma\_0})$, positive	$\sigma_0 > 0$	—
β	via raw_m2log2beta, clamped	[0.01, 0.3]	[1.02, 7.82]
ρ	via raw_mlog2rho2, clamped	[0.01, 0.5]	[0.69, 8.52]
$\boldsymbol{\xi}_0$	direct clamp	$[-1, 1]$ per axis	—
Amp	Amp = softplus(raw_Amp), clamp	(0, 1000]	—

Bounds are enforced two ways: by read-only rejection of an out-of-bounds trial step during optimization, and by a projected clamp applied after each optimizer step.

Remark 3.6 (Restricting C to its effective support)**.** The locality weight $\alpha_i = \exp(-\|\boldsymbol{\xi}_i - \boldsymbol{\xi}_0\|^2/(4\beta^2))$ decays as a Gaussian away from $\boldsymbol{\xi}_0$, so grid points whose weight α_i falls below a small threshold (e.g. 10^{-3}) contribute negligibly to $x^\mathsf{T}Cx$ and may be dropped, shrinking C from (d, d) to (d', d') with $d' \ll d$ (the same restriction is applied to the input so only the retained grid points enter $x^\mathsf{T}Cx$). With threshold 10^{-3} the retained support is the disk $\|\boldsymbol{\xi}_i - \boldsymbol{\xi}_0\| \le 2\beta\sqrt{\ln 1000} \approx 5.25\,\beta$ around $\boldsymbol{\xi}_0$; the retained entries of C still carry gradients, while the support itself is computed from detached parameters and is therefore held fixed (non-differentiable) and stable.

3.5 Kernel gradient with respect to the input, $\nabla_x K$

Choosing the next input (and steering diffusion, Part IV) requires the derivative of the kernel with respect to the input. Treating C as symmetric and constant, σ_0^2 constant, and x' (hence $v_{x'}$) constant, the useful intermediate gradients are

$$\nabla_x v_x = 2\,Cx, \qquad \nabla_x\,(x^\mathsf{T}Cx' + \sigma_0^2) = Cx', \qquad \nabla_x \mathcal{M} = \sqrt{\tfrac{v_{x'}}{v_x}}\,Cx, \tag{9}$$

and the angular derivative $dJ/d\theta = -(\pi - \theta)\sin\theta$.

Proposition 3.7 (Input gradient of the arc-cosine kernel)**.**

$$\nabla_x K(x, x') \;=\; \frac{1}{\pi}\left[(\pi - \theta)\,Cx' \;+\; \sin\theta\,\sqrt{\frac{v_{x'}}{v_x}}\,Cx\right] \;=\; \frac{1}{\pi}\left[(\pi - \theta)\,Cx' \;+\; \sin\theta\,\sqrt{\frac{x'^\mathsf{T}Cx' + \sigma_0^2}{x^\mathsf{T}Cx + \sigma_0^2}}\,Cx\right]. \tag{10}$$

By symmetry ($x \leftrightarrow x'$),

$$\nabla_{x'} K(x, x') = \frac{1}{\pi}\left[(\pi-\theta)\,Cx + \sin\theta\,\sqrt{\frac{x^{\mathsf{T}}Cx+\sigma_0^2}{x'^{\mathsf{T}}Cx'+\sigma_0^2}}\,Cx'\right]. \tag{11}$$

Proof. Write $K = \frac{1}{\pi}\mathcal{M}\,J(\theta)$ and apply the product rule $\nabla_x K \propto (\nabla_x \mathcal{M})J + \mathcal{M}(\nabla_x J)$. Using $\nabla_x J = (dJ/d\theta)\nabla_x\theta$ with $\nabla_x\theta = -(1/\sin\theta)\,\nabla_x(\cos\theta)$ and $dJ/d\theta = -(\pi-\theta)\sin\theta$, the $\sin\theta$ cancels and $\nabla_x J = (\pi-\theta)\,\nabla_x(\cos\theta)$ with $\nabla_x(\cos\theta) = \frac{1}{\mathcal{M}}Cx' - \frac{\cos\theta}{\mathcal{M}}\sqrt{v_{x'}/v_x}\,Cx$ (recall $\cos\theta = (x^{\mathsf{T}}Cx' + \sigma_0^2)/\mathcal{M}$). Collecting terms,

$$\pi\,\nabla_x K = \underbrace{\sin\theta\sqrt{\tfrac{v_{x'}}{v_x}}Cx}_{[A]} + \underbrace{(\pi-\theta)\cos\theta\sqrt{\tfrac{v_{x'}}{v_x}}Cx}_{[B]} + \underbrace{(\pi-\theta)Cx'}_{[C]} - \underbrace{(\pi-\theta)\cos\theta\sqrt{\tfrac{v_{x'}}{v_x}}Cx}_{[D]}.$$

Terms $[B]$ and $[D]$ are identical with opposite sign and cancel, leaving $[A]+[C]$, which is (10). □

Every term carries a factor C, so $\nabla_x K$ lives in the column space of C; this is the algebraic root of the subspace analysis in Part III (gradient ascent on the utility already concentrates on the top C-eigenvectors).

3.6 Kernel gradients with respect to the hyperparameters, $\partial C/\partial\theta$

The M-step (Part II training) maximizes the ELBO over the kernel hyperparameters and needs $\partial C/\partial\theta$, which then chains through $v_x = x^{\mathsf{T}}Cx$ and the cross-term into $\partial K/\partial\theta$ using the same quadratic-form structure as §3.5 with C replaced by $\partial C/\partial\theta$.

Proposition 3.8 (Structured-prior hyperparameter gradients (natural parameters))**.** *Writing* $C = \mathrm{Amp}\,\alpha\,C_{\mathrm{smooth}}\,\alpha^{\mathsf{T}}$*, the amplitude enters linearly and the shape parameters through their Gaussian exponents. Each shape-parameter derivative is* C_{ij} *— which already carries the* Amp *factor — times the derivative of its log-argument, while the amplitude derivative strips one factor of* Amp*:*

$$\begin{aligned}
\frac{\partial C_{ij}}{\partial \mathrm{Amp}} &= \alpha_i\,[C_{\mathrm{smooth}}]_{ij}\,\alpha_j = \frac{C_{ij}}{\mathrm{Amp}},\\
\frac{\partial C_{ij}}{\partial \beta} &= C_{ij}\,\frac{\|\boldsymbol{\xi}_i-\boldsymbol{\xi}_0\|^2 + \|\boldsymbol{\xi}_j-\boldsymbol{\xi}_0\|^2}{2\beta^3},\\
\frac{\partial C_{ij}}{\partial \rho} &= C_{ij}\,\frac{\|\boldsymbol{\xi}_i-\boldsymbol{\xi}_j\|^2}{\rho^3},\\
\nabla_{\boldsymbol{\xi}_0} C_{ij} &= \frac{C_{ij}}{2\beta^2}\,[(\boldsymbol{\xi}_i-\boldsymbol{\xi}_0)+(\boldsymbol{\xi}_j-\boldsymbol{\xi}_0)] \qquad (\textit{2D vector}).
\end{aligned} \tag{12}$$

Proof. For Amp: $C = \mathrm{Amp}\,(\alpha\,C_{\mathrm{smooth}}\,\alpha^{\mathsf{T}})$ is linear in Amp, so $\partial C_{ij}/\partial\mathrm{Amp} = \alpha_i[C_{\mathrm{smooth}}]_{ij}\alpha_j = C_{ij}/\mathrm{Amp}$. For β: $\partial_\beta\big[-(\|\boldsymbol{\xi}_i-\boldsymbol{\xi}_0\|^2 + \|\boldsymbol{\xi}_j-\boldsymbol{\xi}_0\|^2)/(4\beta^2)\big] = +(\|\boldsymbol{\xi}_i-\boldsymbol{\xi}_0\|^2 + \|\boldsymbol{\xi}_j-\boldsymbol{\xi}_0\|^2)/(2\beta^3)$. For ρ: $\partial_\rho\big[-\|\boldsymbol{\xi}_i-\boldsymbol{\xi}_j\|^2/(2\rho^2)\big] = +\|\boldsymbol{\xi}_i-\boldsymbol{\xi}_j\|^2/\rho^3$. For $\boldsymbol{\xi}_0$: $\nabla_{\boldsymbol{\xi}_0}\|\boldsymbol{\xi}-\boldsymbol{\xi}_0\|^2 = 2(\boldsymbol{\xi}_0-\boldsymbol{\xi})$, giving $\nabla_{\boldsymbol{\xi}_0}\log C_{ij} = (1/(2\beta^2))[(\boldsymbol{\xi}_i-\boldsymbol{\xi}_0)+(\boldsymbol{\xi}_j-\boldsymbol{\xi}_0)]$. □

Remark 3.9 (Gradients in the raw parameterization)**.** Because the model optimizes the raw parameters (§3.4), the corresponding *raw*-parameter gradients follow from (12) by the chain rule:

$$\begin{aligned}
\frac{\partial C_{ij}}{\partial(\mathrm{raw_m2log2beta})} &= C_{ij}\,(\log\alpha_i + \log\alpha_j) = -C_{ij}\,\frac{\|\boldsymbol{\xi}_i-\boldsymbol{\xi}_0\|^2 + \|\boldsymbol{\xi}_j-\boldsymbol{\xi}_0\|^2}{4\beta^2},\\
\frac{\partial C_{ij}}{\partial(\mathrm{raw_mlog2rho2})} &= C_{ij}\,\log[C_{\mathrm{smooth}}]_{ij} = -C_{ij}\,\frac{\|\boldsymbol{\xi}_i-\boldsymbol{\xi}_j\|^2}{2\rho^2}.
\end{aligned} \tag{13}$$

and, since $\boldsymbol{\xi}_0$ is a direct parameter, $\partial C_{ij}/\partial \boldsymbol{\xi}_0 = (C_{ij}/(2\beta^2))[(\boldsymbol{\xi}_i - \boldsymbol{\xi}_0) + (\boldsymbol{\xi}_j - \boldsymbol{\xi}_0)]$ coincides with the natural form in (12). The raw and natural gradients are related by the constraint-transform Jacobian $d(\text{raw})/d\beta = -2/\beta$, etc.

3.7 Numerical stability

Three stability mechanisms act at three distinct levels; they must not be conflated.

1. C **matrix — symmetrization only.** $C \leftarrow (C + C^{\mathsf{T}})/2$ cancels the floating-point rounding asymmetry from the outer product. *No diagonal jitter is added to C.*
2. **Kernel evaluation — domain clamps.** With eps $= 10^{-7}$, $\cos\theta$ is clamped into $(-1, 1)$ so arccos is finite, and $1 - \cos^2\theta$ is clamped at min $=$ eps so $\sin\theta = \sqrt{1 - \cos^2\theta}$ never takes the square root of a negative rounding result. These keep the transcendental operations in-domain and are unrelated to Cholesky jitter.
3. **GP covariance / Cholesky — jitter on the gram matrices, not on C.** Jitter is added to the *output* gram matrices of the arc-cosine kernel — the inducing gram $\widetilde{K} = K(\widetilde{Z}, \widetilde{Z})$ and the predictive gram $K(X, X)$ — *not* to C. A pre-Cholesky jitter (default 10^{-4}) is added to $\widetilde{K}$ and $K(X, X)$; the kernel evaluation itself adds none (that would double-jitter). On a failed factorization the gram matrix is promoted to double precision (sequential Cholesky subtractions are prone to catastrophic loss of accuracy in single precision) and the factorization is retried with escalating jitter $10^{-4}, 10^{-3}, 10^{-2}$ (up to three tries).

Caveat. Some intuition expects diagonal jitter on the prior C; there is none. C receives only symmetrization. Jitter lives exclusively on the GP gram matrices $\widetilde{K}/K(X, X)$. Do not "add jitter to C" thinking it is missing.

Part II
Inference: variational learning of the latent

4 Sparse variational Gaussian-process inference

Part I fixed the generative model: a zero-mean latent Gaussian process $\lambda \sim \mathcal{GP}(0, K)$ over input space, an exponential link to the rate $f(x) = \exp(A\,\lambda(x) + \lambda_0)$, and Poisson counts $y_i \sim \text{Poisson}(f(x_i))$, where K is the first-order arc-cosine kernel of (2) built on the structured spatial prior covariance C of (6). Here we turn that model into a trainable posterior. The variational posterior developed below can be represented equivalently in the full inducing space or in the eigenbasis of §5.

4.1 Poisson breaks conjugacy

Given a batch of inputs $X = (x_1, \ldots, x_N)$ and counts $y = (y_1, \ldots, y_N)$, the exact posterior over the latent and the marginal likelihood,

$$p(\lambda \mid y, X, \theta) \;\propto\; \Big[\textstyle\prod_i \text{Poisson}\big(y_i \mid e^{A\lambda_i + \lambda_0}\big)\Big]\, \mathcal{N}(\lambda \mid 0, K_\theta),$$
$$p(y \mid X, \theta) = \int \prod_i \text{Poisson}\big(y_i \mid e^{A\lambda_i + \lambda_0}\big)\, \mathcal{N}(\lambda \mid 0, K_\theta)\, d\lambda, \tag{14}$$

have no closed form: the exponential-Poisson likelihood is not conjugate to the Gaussian prior, so the integral over λ is intractable. We therefore approximate the posterior by a tractable Gaussian and maximise a variational lower bound (§4.4).

Remark 4.1 (The conjugate contrast)**.** For a *Gaussian* observation $y = \lambda + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma_r^2 I)$, everything would be closed-form: the marginal is $y \mid X \sim \mathcal{N}(0, K + \sigma_r^2 I)$, the predictive at a test input x_* is Gaussian with mean $k_*^\mathsf{T}(K+\sigma_r^2 I)^{-1}y$ and variance $K(x_*, x_*) - k_*^\mathsf{T}(K+\sigma_r^2 I)^{-1}k_*$, and the log-evidence is $-\frac{1}{2}y^\mathsf{T}(K + \sigma_r^2 I)^{-1}y - \frac{1}{2}\log|K + \sigma_r^2 I| - \frac{N}{2}\log 2\pi$. None of these survive the Poisson link; the variational route below recovers a usable posterior in its place.

4.2 Inducing points and the variational posterior

Definition 4.2 (Sparse variational posterior)**.** Introduce M *inducing points* $\widetilde{Z} = \{\widetilde{z}_1, \ldots, \widetilde{z}_M\}$ with latent values $\widetilde{\lambda} = \lambda(\widetilde{Z})$, and place a Gaussian variational posterior on them,

$$q(\widetilde{\lambda}) = \mathcal{N}(\widetilde{\lambda} \mid m, V), \qquad m \in \mathbb{R}^M, \quad V \in \mathbb{S}_{++}^M. \tag{15}$$

The posterior at *any* input x is recovered by marginalising the conditional prior $p(\lambda(x) \mid \widetilde{\lambda})$ against $q(\widetilde{\lambda})$.

Exact inference would need the $N\times N$ kernel over all training inputs, an $O(N^3)$ cost; the inducing construction reduces it to $O(NM^2)$.

Remark 4.3 (Active-learning invariant $M = N$)**.** In the active-learning setting the inducing set is taken equal to the training set, $X = \widetilde{Z}$, so $M = N$ and $K = \widetilde{K}$. The "sparse" posterior is then *exact*. The SVGP formalism is retained not for sparsity but for two other reasons: (i) it supplies the closed-form variational machinery of §4.3–§4.5, and (ii) its inducing structure admits an $O(M)$ rank-one online growth that appends one input per active-learning step.

4.3 Projected posterior moments

Marginalising $p(\lambda(x) \mid \widetilde{\lambda})\, q(\widetilde{\lambda})$ yields a Gaussian $q(\lambda(x)) = \mathcal{N}\big(\mu(x), \sigma^2(x)\big)$ whose moments are the inducing-point moments projected through the kernel:

$$\begin{aligned} \mu(x) &= k(x)^\mathsf{T}\, \widetilde{K}^{-1}\, m, \\ \sigma^2(x) &= K(x,x) + k(x)^\mathsf{T}\, \widetilde{K}^{-1}(V - \widetilde{K})\widetilde{K}^{-1}\, k(x), \end{aligned} \tag{16}$$

with the cross-covariance vector $k(x) = K(\widetilde{Z}, x) \in \mathbb{R}^M$, the inducing gram $\widetilde{K} = K(\widetilde{Z}, \widetilde{Z}) \in \mathbb{R}^{M\times M}$, and the prior self-variance $K(x,x)$ given by the self-kernel identity (5). Introducing the *projection vector* $u(x) = \widetilde{K}^{-1}k(x)$ and the Schur complement $s(x) = K(x,x) - k(x)^\mathsf{T}\widetilde{K}^{-1}k(x)$, the variance splits into a data-independent residual and an epistemic term,

$$\sigma^2(x) = \underbrace{\big[K(x,x) - k(x)^\mathsf{T}\widetilde{K}^{-1}k(x)\big]}_{\text{residual (Nyström/Schur) } s(x)} + \underbrace{k(x)^\mathsf{T}\widetilde{K}^{-1}V\widetilde{K}^{-1}k(x)}_{\text{epistemic}} = s(x) + u(x)^\mathsf{T} V\, u(x), \tag{17}$$

and the mean reads $\mu(x) = u(x)^\mathsf{T} m$. The two forms of σ^2 are algebraically identical since $k^\mathsf{T}\widetilde{K}^{-1}(V - \widetilde{K})\widetilde{K}^{-1}k = k^\mathsf{T}\widetilde{K}^{-1}V\widetilde{K}^{-1}k - k^\mathsf{T}\widetilde{K}^{-1}k$.

Remark 4.4 (Prior sanity check)**.** At the prior $V = \widetilde{K}$ the correction in (16) vanishes and $\sigma^2(x) = K(x,x)$ (the prior variance); as data drive V below $\widetilde{K}$, $\sigma^2(x) < K(x,x)$.

These moments can be evaluated directly in the M-dimensional inducing space or, equivalently, in the eigenbasis of $\widetilde{K}$ (§5.3).

4.4 The ELBO

All parameters are learned by maximising the evidence lower bound on $\log p(y)$,

$$\mathcal{L} \;=\; \sum_i \mathbb{E}_{q(\lambda_i)}\big[\log \mathrm{Poisson}(y_i \mid \lambda_i)\big] \;-\; \mathrm{KL}\big(q(\widetilde{\lambda}) \,\|\, p(\widetilde{\lambda})\big), \tag{18}$$

which is tight when q equals the true posterior. Because the likelihood factorises over inputs and $q(\lambda_i) = \mathcal{N}(\mu_i, \sigma_i^2)$ is Gaussian, each expected log-likelihood term is closed-form:

$$\mathbb{E}_{q(\lambda_i)}\big[\log \mathrm{Poisson}(y_i \mid \lambda_i)\big] \;=\; y_i(A\mu_i + \lambda_0) \;-\; \exp\big(A\mu_i + \tfrac{1}{2}A^2\sigma_i^2 + \lambda_0\big) \;+\; \mathrm{const}, \tag{19}$$

with $\mathrm{const} = -\log(y_i!)$ independent of all parameters. The KL term is the standard Gaussian–Gaussian divergence between $q(\widetilde{\lambda}) = \mathcal{N}(m, V)$ and the prior $p(\widetilde{\lambda}) = \mathcal{N}(0, \widetilde{K})$,

$$\mathrm{KL}(q \,\|\, p) \;=\; \tfrac{1}{2}\Big[\, \log \frac{|\widetilde{K}|}{|V|} \;+\; \mathrm{Tr}(\widetilde{K}^{-1}V) \;+\; m^{\mathsf{T}}\widetilde{K}^{-1}m \;-\; M \,\Big]. \tag{20}$$

Remark 4.5 (The $-M$ constant)**.** The $-M$ in (20) is parameter-independent (it is the dimension of the inducing Gaussian) and carries zero gradient, so it affects only the reported loss magnitude, never the optimum or the predictions. The eigenspace evaluation of §5.4 carries the analogous $-n_b$ for the same reason.

The full objective is maximised by EM-style coordinate ascent (E-step on (m, V), F-step on (A, λ_0), M-step on the kernel hyperparameters); the update equations are derived in the training-algorithm section. Equation (18) is the single objective all three blocks share.

4.5 Expected rate via the Gaussian MGF

The exponential term in (19) is the *expected rate* under q,

$$\bar{f}_i \;=\; \mathbb{E}_{q(\lambda_i)}\big[\exp(A\lambda_i + \lambda_0)\big] \;=\; \exp\big(A\mu_i + \tfrac{1}{2}A^2\sigma_i^2 + \lambda_0\big). \tag{21}$$

This is the Gaussian moment-generating function $\mathbb{E}[e^{tZ}] = \exp(t\mu + \frac{1}{2}t^2\sigma^2)$ applied to $Z = A\lambda_i + \lambda_0$ at $t = 1$.

Remark 4.6 (Jensen inflation)**.** The term $\frac{1}{2}A^2\sigma_i^2$ is strictly positive whenever $\sigma_i^2 > 0$, so posterior *uncertainty raises* the predicted rate above the point estimate $\exp(A\mu_i + \lambda_0)$ — a direct consequence of Jensen's inequality for the convex map exp. Confidence and rate are thus coupled: a poorly-constrained input is predicted to have a *higher* rate, not lower.

4.6 Prediction: expected count

The moments (16) do not depend on the observed y, so the same formulas serve as the predictive distribution at any novel input x_*. The expected count is the expected rate (21) evaluated there,

$$\langle y_* \rangle = \langle f(x_*) \rangle = \exp\big(A\mu(x_*) + \tfrac{1}{2}A^2\sigma^2(x_*) + \lambda_0\big), \tag{22}$$

with $\mu(x_*), \sigma^2(x_*)$ from (16) using freshly evaluated $k(x_*)$ and $K(x_*, x_*)$. The rate is log-normal, so its variance is

$$\mathrm{Var}(f(x)) = \langle f(x) \rangle^2 \big(e^{A^2\sigma^2(x)} - 1\big). \tag{23}$$

4.7 The posterior cross-covariance pitfall

The self-variance of (16) extends formally to the *cross-covariance* between two distinct inputs x (a candidate sample location) and x_* (a query location):

$$\Sigma_{x,x_*} = \mathrm{Cov}\big[\lambda(x), \lambda(x_*) \mid \mathcal{D}\big] = K(x, x_*) + u(x)^\mathsf{T}(V - \widetilde{K})u(x_*), \qquad u(\cdot) = \widetilde{K}^{-1}k(\cdot), \tag{24}$$

which reduces to $\sigma^2(x)$ at $x = x_*$.

Remark 4.7 (Failure outside the inducing hull)**.** For a query x_* *outside* the inducing region, (24) produces cross-covariances that violate the Cauchy–Schwarz bound $|\Sigma_{x,x_*}| \le \sqrt{\Sigma_{x,x}\,\Sigma_{x_*,x_*}}$. Measured example (RBF, $\ell = 0.2$, 20 inducing points in $[-1, 1]$, sample $x = 0$): at $x_* = \pm 1.5$ the manual formula gives $\Sigma \approx -4$ while the consistent joint covariance gives ≈ 0; with self-variances ~ 0.04 the valid maximum is ~ 0.04, so -4 is two orders of magnitude out of range. Root cause: in the residual $K(x, x_*) - u(x)^\mathsf{T}\widetilde{K}\,u(x_*)$ the direct kernel $K(x, x_*) \to 0$ (finite lengthscale) outside the hull, but $u(x_*) = \widetilde{K}^{-1}k(x_*)$ can have large entries through $\widetilde{K}^{-1}$, and the two pieces fail to cancel.

Left unchecked, a value like $\Sigma_{x,x_*} \approx -4$ feeds the conditional moments $\mu_{\mathrm{cond}}(x_*) = \mu(x_*) + (\Sigma_{x,x_*}/\Sigma_{x,x})(\lambda_{\mathrm{obs}} - \mu(x))$ and $\sigma^2_{\mathrm{cond}}(x_*) = \Sigma_{x_*,x_*} - \Sigma^2_{x,x_*}/\Sigma_{x,x}$ of the distribution-aware utility (Part III) with a regression coefficient ≈ -100, giving nonsensical conditional means and spurious utility peaks at the edges of the inducing region. The remedy is to never assemble the cross-block from the manual formula (24); instead read the cross-block from the jointly-formed covariance of (x, x_*), which is Nyström-consistent — it carries the conditional correction term that (24) drops outside the hull. The pathology is therefore confined to hand-assembled cross-covariances; any computation that forms a cross-covariance from (24) outside the inducing hull will reintroduce it.

4.8 Whitening

An alternative reparameterisation is *Cholesky whitening*, which makes the prior a standard normal. With the Cholesky factor $\widetilde{K} = L_{\widetilde{K}} L_{\widetilde{K}}^\mathsf{T}$, define the whitened inducing values

$$\widetilde{\widetilde{\lambda}} = L_{\widetilde{K}}^{-1}(\widetilde{\lambda} - \mu_z) \;\sim\; \mathcal{N}(0, I) \quad \text{under the prior,} \tag{25}$$

where μ_z is the prior mean at the inducing points. Optimisation conditions much better in these coordinates. In these coordinates one stores the *whitened* variational parameters $(\widetilde{m}, \widetilde{V})$, related to the actual ones by

$$m - \mu_z = L_{\widetilde{K}}\,\widetilde{m}, \quad V = L_{\widetilde{K}}\,\widetilde{V}\,L_{\widetilde{K}}^\mathsf{T}; \qquad \widetilde{m} = L_{\widetilde{K}}^{-1}(m - \mu_z), \quad \widetilde{V} = L_{\widetilde{K}}^{-1} V\, L_{\widetilde{K}}^{-\mathsf{T}}. \tag{26}$$

The whitened mean thus encodes the *deviation* from the prior mean function, not absolute function values. Recovering the moments from the whitened parameters therefore requires three corrections: unwhiten the covariance ($V = L_{\widetilde{K}}\widetilde{V}L_{\widetilde{K}}^\mathsf{T}$), unwhiten the mean ($m - \mu_z = L_{\widetilde{K}}\widetilde{m}$), and add the mean function back, $\mu_* = \mu(x_*) + u(x_*)^\mathsf{T}L_{\widetilde{K}}\widetilde{m}$ and $\sigma_*^2 = s(x_*) + u(x_*)^\mathsf{T}L_{\widetilde{K}}\widetilde{V}L_{\widetilde{K}}^\mathsf{T}u(x_*)$.

Remark 4.8 (Zero prior mean)**.** With a zero prior mean, $\mu_z = 0$ and the "add the mean function" step is trivially zero; the whitening maps (26) themselves apply for any prior mean.

5 The eigenspace representation

The variational posterior $q(\widetilde{\lambda})$ of §4 can be represented equivalently in the full M-dimensional inducing space or, more economically, in the eigenbasis of the inducing gram $\widetilde{K}$. This section develops the eigenbasis form: the same prior $\mathcal{N}(0, \widetilde{K})$, the same arc-cosine kernel, the same Poisson(A, λ_0) likelihood, the same expected rate (21), and the same ELBO (18), re-expressed in a low-dimensional basis in which $\widetilde{K}$ is diagonal.

5.1 Eigendecomposition of the inducing gram

The inducing gram $\widetilde{K}$ ($M \times M$) has effective rank far below M (typically $\sim$ 10–50). We eigendecompose it and keep only the well-conditioned directions:

$$\begin{aligned} \widetilde{K} = B\,\Lambda\,B^{\mathsf{T}}, \qquad & \Lambda = \mathrm{diag}(\mathrm{eigvals}_b), \\ B \in \mathbb{R}^{M \times n_b}, \qquad & B^{\mathsf{T}}B = I_{n_b}, \end{aligned} \tag{27}$$

where the kept set is chosen by a *relative* threshold with an absolute floor,

$$\text{keep eigenpair } j \iff \Lambda_j > \max\big(\Lambda_{\max} \cdot \texttt{tol},\ \texttt{tol}\big), \qquad \texttt{tol} = 10^{-4}. \tag{28}$$

The eigenvalues are taken in ascending order; B collects the kept eigenvectors as columns and Λ the kept eigenvalues. The kept count n_b is *dynamic*: it shifts across training iterations as the kernel hyperparameters move, and grows by ~ 1 when a point is appended. Typical $n_b \approx$ 10–11 in the active-learning setting, up to ~ 50 in larger problems.

Three payoffs motivate the projection: (i) dimensionality drops $M \to n_b$, so per-step cost falls $O(M^3) \to O(n_b^3)$; (ii) $\widetilde{K}$ becomes *diagonal* in the new basis, $\widetilde{K}_b = \Lambda$, so $\widetilde{K}^{-1}$ is the element-wise reciprocal $\mathrm{diag}(1/\Lambda)$ — no matrix solve; (iii) dropping the small eigenvalues gives implicit regularisation.

5.2 The eigenspace state

The eigenspace posterior is carried together with the precomputed kernel projections used below:

symbol	shape	meaning
m_b	(n_b)	variational mean in the eigenbasis, $m_b = B^{\mathsf{T}}m$
V_b	(n_b, n_b)	variational covariance in the eigenbasis, $V_b = B^{\mathsf{T}}VB$ — **dense**
B	(M, n_b)	eigenvector matrix of $\widetilde{K}$ (the basis)
Λ	(n_b)	kept eigenvalues of $\widetilde{K}$ (ascending)
$\widetilde{K}_b$	(n_b, n_b)	inducing gram in the eigenbasis = $\mathrm{diag}(\Lambda)$ — **diagonal**
K_b	(N, n_b)	cross-kernel projected into the eigenbasis, $K_b = K(X, \widetilde{Z})\,B$
a	(N, n_b)	projection $K\widetilde{K}^{-1}$ in the eigenbasis, $a = K_b/\Lambda$ (element-wise)
Kvec	(N)	diagonal self-kernel $K(x_i, x_i)$, the per-point prior variance

Remark 5.1 (Only $\widetilde{K}_b$ is diagonal)**.** V_b is a *dense* $n_b \times n_b$ matrix even though $\widetilde{K}_b$ is diagonal; never assume a diagonal V_b. The projection row $a_i = (K_b/\Lambda)_i$ is exactly $k(x_i)^{\mathsf{T}}\widetilde{K}^{-1}$ expressed in the eigenbasis, obtained by element-wise division rather than a matrix solve.

Initialisation sets the variational parameters to the prior:

$$m_b = 0, \qquad V_b = \widetilde{K}_b \quad \text{(i.e. } m = 0,\ V = \widetilde{K},\ \text{so KL} = 0 \text{ at start)}. \tag{29}$$

5.3 Posterior moments in the eigenbasis

With $a = K_b/\Lambda$ (element-wise), the projected moments (16) at the training points become:

$$\begin{aligned} \mu &= a\, m_b, \\ \sigma^2 &= \text{Kvec} + \text{diag}\big(a\,(V_b - \widetilde{K}_b)\,a^{\mathsf{T}}\big) = \text{Kvec} + \text{rowsum}\big(a \odot (a\,(V_b - \widetilde{K}_b))\big), \\ \sigma^2 &\leftarrow \max(\sigma^2,\ 10^{-6}), \end{aligned} \tag{30}$$

where μ, σ^2 are the per-training-point posterior-moment vectors and $\odot$ is the element-wise product; the two variance expressions coincide because $\text{rowsum}(a \odot (aM)) = \text{diag}(aMa^{\mathsf{T}})$ for any M. Term by term this equals (16): $\text{Kvec}_i = K(x_i, x_i)$ (the self-kernel identity (5)) and $a_i(V_b - \widetilde{K}_b)a_i^{\mathsf{T}} = k(x_i)^{\mathsf{T}}\widetilde{K}^{-1}(V - \widetilde{K})\widetilde{K}^{-1}k(x_i)$. For **test** points the same two expressions run on a freshly projected $a = K(x_*, \widetilde{Z})B/\Lambda$ with $\text{Kvec} = K(x_*, x_*)$, so no training-point kernel is recomputed; the predicted rate there is $f_{\text{pred}} = \exp(A\mu + \frac{1}{2}A^2\sigma^2 + \lambda_0)$, i.e. (21).

5.4 ELBO and KL in the eigenbasis

The eigenspace ELBO uses the log-likelihood (19) and, because $\widetilde{K}_b = \text{diag}(\Lambda)$, the KL (20) collapses to sums over the kept dimensions:

$$\begin{aligned} \text{KL}(q \,\|\, p) &= \tfrac{1}{2}\Big[\, \text{Tr}(\widetilde{K}^{-1}V) + m^{\mathsf{T}}\widetilde{K}^{-1}m - n_b + \log|\widetilde{K}| - \log|V| \,\Big] \\ &= \tfrac{1}{2}\Big[\, \textstyle\sum_j \dfrac{V_b[j,j]}{\Lambda_j} \;+\; \sum_j \dfrac{m_b[j]^2}{\Lambda_j} \;-\; n_b \;+\; \sum_j \log\Lambda_j \;-\; \log\det V_b \,\Big]. \end{aligned} \tag{31}$$

Remark 5.2 (Diagonal-trace assumption)**.** The trace term uses only $\text{diag}(V_b)$, which is valid *only because* $\widetilde{K}_b$ is diagonal in the basis B in which the ELBO is evaluated — i.e. when the kernel and the basis are mutually consistent. The $\log|V|$ term uses the full log-determinant of the dense V_b. The $-n_b$ is a zero-gradient constant (cf. the $-M$ remark after (20)): it shifts only the reported loss magnitude.

6 The training algorithm: EM-style coordinate ascent

The model assembled in Parts I–II has three groups of free parameters, and all three sit on the *single* evidence lower bound $\mathcal{L}$ of (18):

(a) the **variational moments** m_b, V_b of the eigenspace posterior $q(\widetilde{\lambda}) = \mathcal{N}(m, V)$;

(b) the **rate parameters** of the Poisson likelihood, the gain A and the offset λ_0, defining $f = \exp(A\lambda + \lambda_0)$;

(c) the **kernel hyperparameters** $\{\sigma_0, \text{Amp}, \beta, \rho, \varepsilon_{0x}, \varepsilon_{0y}\}$ of the structured arc-cosine prior (Part I).

Training maximises the one bound $\mathcal{L}$ by **block coordinate ascent**: each step improves one block while holding the other two fixed. The three steps are the E-step (variational moments), the F-step (rate parameters), and the M-step (kernel hyperparameters), run once per outer iteration in a fixed order (§6.1).

6.1 Overview: three coordinate blocks, one ELBO, one loop

Each step optimises one block and freezes the rest:

Step	Free block	Held fixed	Method
E	m_b, V_b	kernel; A, λ_0; B	closed-form Newton (no automatic differentiation)
F	A (λ_0 closed-form)	m_b, V_b (via μ, σ^2); kernel	LBFGS on $\log A$, or damped Newton
M	$\{\sigma_0, \mathrm{Amp}, \beta, \rho, \boldsymbol{\xi}_0\}$	m_b, V_b; A, λ_0; B	LBFGS + gradients (analytical or automatic)

(In the M-row, the centre $\boldsymbol{\xi}_0 = (\varepsilon_{0x}, \varepsilon_{0y})$ of the localized region counts as the two coordinate hyperparameters, so the free block has six scalars.)

Loop order (one outer iteration). Per outer iteration:

1. **Re-sync the eigenbasis.** Rebuild the eigenbasis B from the hyperparameters the *previous* M-step moved, then refresh $\mu, \sigma^2, \bar{f}$ — this is where the M-step's deferred B update lands.
2. **E-step** — one or more closed-form Newton steps on m_b, V_b (§6.2).
3. **F-step** — fit A (and closed-form λ_0), unless the F-step is interleaved into the E-loop (§6.3).
4. **Metrics + ELBO + early stopping** — computed *deliberately before* the M-step.
5. **M-step** — update the kernel hyperparameters (§6.4); the B update is *deferred* to step 1 of the next iteration.

Remark 6.1 (Why metrics precede the M-step)**.** The M-step changes the kernel parameters *without* recomputing the eigenbasis B (that is deferred, for cost and stability). Every logged quantity — the ELBO, the early-stopping metric — must therefore be read while the model state is still internally consistent, i.e. *before* the M-step perturbs the kernel. Early stopping is on the ELBO, with patience and best-restore.

6.2 The E-step: a Newton update of the variational posterior

At *fixed* kernel and fixed A, λ_0, the E-step maximises the ELBO (18) over the variational moments. Writing the inducing-space posterior $q(\widetilde{\lambda}) = \mathcal{N}(m, V)$ against the prior $p(\widetilde{\lambda}) = \mathcal{N}(0, \widetilde{K})$,

$$\mathcal{L}(m, V) = \underbrace{\sum_i \mathbb{E}_{q(\lambda_i)}[\log \mathrm{Poisson}(y_i \mid \lambda_i)]}_{\mathcal{L}_{\mathrm{data}}} - \mathrm{KL}(q(\widetilde{\lambda}) \,\|\, p(\widetilde{\lambda})), \tag{32}$$

with the projected posterior moments μ_i, σ_i^2 of (16) and the KL contributing, through the E-step variables, $-\frac{1}{2} m^\top \widetilde{K}^{-1} m$ (couples to m) and $\frac{1}{2}\log|V| - \frac{1}{2}\mathrm{Tr}(\widetilde{K}^{-1} V)$ (couples to V).

Poisson expected log-likelihood and its moment derivatives. The data term is closed-form via the Gaussian moment-generating function, with the expected rate $\bar{f}_i$ of (21):

$$\mathcal{L}_{\mathrm{data}} = \sum_i \left[\, y_i(A\mu_i + \lambda_0) - \bar{f}_i \,\right], \qquad \bar{f}_i = \exp(A\mu_i + \tfrac{1}{2}A^2\sigma_i^2 + \lambda_0), \tag{33}$$

(the $-\log y_i!$ constant is dropped). The three point-derivatives that assemble the Newton step follow by the chain rule on $\bar{f}_i$:

$$\frac{\partial \bar{f}_i}{\partial \mu_i} = A\,\bar{f}_i, \qquad \frac{\partial \bar{f}_i}{\partial \sigma_i^2} = \tfrac{1}{2}A^2\,\bar{f}_i, \qquad \frac{\partial}{\partial \mu_i}\,\mathbb{E}_q[\log \text{Poisson}] = A(y_i - \bar{f}_i). \tag{34}$$

The two Newton helpers. Accumulating those derivatives through the projection $\mu_i = k_i^\mathsf{T}\widetilde{K}^{-1}m$ defines the gradient helper g_E and the (Poisson-Fisher) curvature G_E:

$$g_\mathrm{E} \;=\; A\sum_i k_i(y_i - \bar{f}_i) \;\in \mathbb{R}^M, \qquad G_\mathrm{E} \;=\; A^2\sum_i \bar{f}_i\, k_i k_i^\mathsf{T} \;\in \mathbb{R}^{M\times M}. \tag{35}$$

g_E is the kernel-projected mismatch between observed and expected counts; G_E is the Poisson Fisher information and is PSD because every $\bar{f}_i > 0$. (Note g_E carries the subscript E specifically to keep it distinct from the log-rate $g = A\lambda + \lambda_0$ of the utility part.)

The V-update (closed form). With μ_i independent of V, setting $\nabla_V\mathcal{L} = 0$ gives $V^{-1} = \widetilde{K}^{-1}(\widetilde{K} + G_\mathrm{E})\widetilde{K}^{-1}$, hence

$$\boxed{V \;=\; \widetilde{K}\,(\widetilde{K} + G_\mathrm{E})^{-1}\,\widetilde{K}} \tag{36}$$

— a *symmetric* sandwich of two symmetric-PD factors, so V is a valid covariance. This symmetry is the substance of the "corrected" E-step: the superseded form $V = (\widetilde{K} + G_\mathrm{E})^{-1}\widetilde{K}$ is a product of two non-commuting symmetric matrices and is therefore *not* symmetric (Cholesky/PSD failures).

The m-update. The exact Newton step $m \leftarrow m - H_m^{-1}\nabla_m\mathcal{L}$, with $\nabla_m\mathcal{L} = \widetilde{K}^{-1}(g_\mathrm{E} - m)$ and Hessian $H_m = -\widetilde{K}^{-1}(\widetilde{K} + G_\mathrm{E})\widetilde{K}^{-1}$, gives the corrected form $m \leftarrow m + \widetilde{K}(\widetilde{K} + G_\mathrm{E})^{-1}(g_\mathrm{E} - m)$. The map actually used is *not* this; it is the algebraically older

$$\boxed{m \;\leftarrow\; \widetilde{K}\,(\widetilde{K} + G_\mathrm{E})^{-1}(G_\mathrm{E}\,\widetilde{K}^{-1}m + g_\mathrm{E})} \tag{37}$$

(equal to $V_\text{new}(G_\mathrm{E}\widetilde{K}^{-1}m + g_\mathrm{E})$ with V_new from (36)). The two maps differ only in the first term $(\widetilde{K}(\widetilde{K} + G_\mathrm{E})^{-1}G_\mathrm{E}\widetilde{K}^{-1}m$ vs. $G_\mathrm{E}(\widetilde{K} + G_\mathrm{E})^{-1}m)$; they coincide iff $\widetilde{K}$ and G_E commute.
Caveat. The m-update map (37) differs *per iteration* from the corrected Newton step whenever $\widetilde{K}$ and G_E do not commute, but both maps share the same fixed point $m^\star = g_\mathrm{E}$, so the *converged* posterior is identical — the discrepancy is a transient path difference, not a different solution. It is left as-is (it converges in practice). The V-update (36) is unambiguously correct, and it is V that the active-learning variance estimates depend on.

Remark 6.2 (Fixed-point equivalence)**.** Let $T_\mathrm{N}(m) = m + \widetilde{K}(\widetilde{K} + G_\mathrm{E})^{-1}(g_\mathrm{E} - m)$ (corrected) and $T_\mathrm{C}(m) = \widetilde{K}(\widetilde{K} + G_\mathrm{E})^{-1}(G_\mathrm{E}\widetilde{K}^{-1}m + g_\mathrm{E})$ (older map). At $m = g_\mathrm{E}$: $T_\mathrm{N}(g_\mathrm{E}) = g_\mathrm{E}$, and $T_\mathrm{C}(g_\mathrm{E}) = \widetilde{K}(\widetilde{K} + G_\mathrm{E})^{-1}(G_\mathrm{E}\widetilde{K}^{-1} + I)g_\mathrm{E} = \widetilde{K}(\widetilde{K} + G_\mathrm{E})^{-1}(G_\mathrm{E} + \widetilde{K})\widetilde{K}^{-1}g_\mathrm{E} = \widetilde{K}\widetilde{K}^{-1}g_\mathrm{E} = g_\mathrm{E}$. Both are valid fixed-point iterations onto the same stationary condition $m^\star = g_\mathrm{E} = A\sum_i k_i(y_i - \bar{f}_i)$.

6.2.1 Why the update is cheap: the eigenspace realisation

The E-step never touches the M-dimensional inducing space directly. It works in the eigenbasis B of $\widetilde{K}$, keeping only the n_b leading directions (eigenvalues above $\max(\lambda_\text{max}\cdot\text{tol}, \text{tol})$, $\text{tol} = 10^{-4}$; in practice $n_b \approx 10$–11 while M may be large). With $\widetilde{K} = B\Lambda B^\mathsf{T}$ the projected gram $\widetilde{K}_b = B^\mathsf{T}\widetilde{K}B =$

diag(Λ) is *diagonal*, so $\widetilde{K}^{-1}$ is trivial. Using the eigenspace projection $a = K\widetilde{K}^{-1}$ (shape $N \times n_b$), one forms the *transformed* helpers:

$$g_{\mathrm{E},b} = A\, a^{\mathsf{T}}(y - \bar{f}) \;=\; \widetilde{K}^{-1} g_{\mathrm{E}} \qquad \text{(eigenbasis coords)}, \tag{38}$$

$$G_{\mathrm{E},b} = A^2\, a^{\mathsf{T}}(\bar{f} \odot a) \;=\; \widetilde{K}^{-1} G_{\mathrm{E}} \widetilde{K}^{-1}, \tag{39}$$

and applies (36)–(37) entirely in the tiny n_b-space:

$$V_b \leftarrow (I + \widetilde{K}_b\, G_{\mathrm{E},b})^{-1} \widetilde{K}_b, \qquad m_b \leftarrow V_b(G_{\mathrm{E},b}\, m_b + g_{\mathrm{E},b}), \qquad V_b \leftarrow \tfrac{1}{2}(V_b + V_b^{\mathsf{T}}). \tag{40}$$

The first line equals (36) exactly, via the identity $(I + PQ^{-1})^{-1} = Q(Q+P)^{-1}$ with $P = G_{\mathrm{E}}, Q = \widetilde{K}$: $(I + \widetilde{K}_b G_{\mathrm{E},b})^{-1}\widetilde{K}_b = (I + G_{\mathrm{E}}\widetilde{K}^{-1})^{-1}\widetilde{K} = \widetilde{K}(\widetilde{K} + G_{\mathrm{E}})^{-1}\widetilde{K}$. Prior initialisation is $m_b = 0$, $V_b = \widetilde{K}_b = \Lambda$. The dimensional collapse $M \to n_b \approx 10$ — not any special-casing — is what makes the Newton step cheap.

6.2.2 Iteration, damping, and the divergence guard

The E-step performs a *single, undamped, closed-form* Newton step; iteration and stabilisation are handled around it:

- **Coupling variable $\bar{f}$.** After each step the posterior is recomputed and $\bar{f} = \exp(A\mu + \frac{1}{2}A^2\sigma^2 + \lambda_0)$ refreshed, then fed into the next step's $g_{\mathrm{E},b}, G_{\mathrm{E},b}$. A, λ_0 are held fixed during the E-step.
- **Revert-on-divergence guard (the effective damping).** Each iteration snapshots (m_b, V_b); if the post-step $\bar{f}$ mean or max exceeds a threshold, or $\bar{f}$ has NaNs, it *reverts* (m_b, V_b) (and A, λ_0 if interleaved), recomputes $\bar{f}$, and breaks the loop.
- **Optional interleaved F-step.** With interleaving enabled, a *damped* Newton update of (A, λ_0) (§6.3, $\alpha = 0.25$) runs after every E-iteration, so the rate parameters track (m_b, V_b) as they move.

6.3 The F-step: fitting the rate function

Here "F" denotes the **rate function** f: the F-step fits the Poisson-rate parameters A, λ_0 that define $\bar{f} = \exp(A\mu + \frac{1}{2}A^2\sigma^2 + \lambda_0)$, holding the variational moments (through frozen μ, σ^2) and the kernel fixed. Here A is the only searched variable; λ_0 is solved in closed form for the current A.

Closed-form λ_0 given A. From $\partial\mathcal{L}/\partial\lambda_0 = 0$ on $\sum_i[y_i\lambda_0 - \exp(\lambda_0)\exp(A\mu_i + \frac{1}{2}A^2\sigma_i^2)]$:

$$\boxed{\lambda_0 \;=\; \log\Big(\textstyle\sum_i y_i\Big) \;-\; \log\Big(\textstyle\sum_i \exp\big(A\mu_i + \frac{1}{2}A^2\sigma_i^2\big)\Big)} \tag{41}$$

(requires $\sum_i y_i > 0$; a zero-count target is a data problem). It is re-evaluated whenever A changes.

LBFGS on A with an analytic gradient. The F-step runs LBFGS over $\log A$, re-solving λ_0 at each evaluation and using the closed-form derivative, with μ, σ^2 the frozen inputs:

$$\frac{\partial\mathcal{L}}{\partial A} = \sum_i \left[y_i\mu_i - (\mu_i + A\sigma_i^2)\,\bar{f}_i\right], \qquad \frac{\partial\mathcal{L}}{\partial(\log A)} = A\frac{\partial\mathcal{L}}{\partial A}. \tag{42}$$

Since LBFGS minimises $-\mathcal{L}$, the gradient it consumes with respect to $\log A$ is $-A\,\partial\mathcal{L}/\partial A$. Guards reject a trial step whose λ_0 over/underflows or whose $\bar{f}$ exceeds the mean/max thresholds, and revert (A, λ_0) if the final λ_0 overflows.

Interleaved joint damped Newton on (A, λ_0). With interleaving enabled, a joint damped Newton step on $\psi = [A, \lambda_0]$ runs *inside* the E-loop. With per-input $f_i = \exp(A\mu_i + \frac{1}{2}A^2\sigma_i^2 + \lambda_0)$ and $d_i = \mu_i + A\sigma_i^2$:

$$R = \begin{bmatrix} \sum_i (y_i\mu_i - d_i f_i) \\ \sum_i (y_i - f_i) \end{bmatrix}, \quad H = -\begin{bmatrix} \sum_i (\sigma_i^2 f_i + d_i^2 f_i) & \sum_i d_i f_i \\ \sum_i d_i f_i & \sum_i f_i \end{bmatrix}, \quad \psi \leftarrow \psi - \alpha\, H^{-1} R, \;\; \alpha = 0.25. \tag{43}$$

H is negative-definite (the log-likelihood is concave in ψ); the damping $\alpha = 0.25$ and the same $\bar{f}$-threshold rejections stabilise the step.

6.4 The M-step: updating the kernel hyperparameters

The M-step maximises the ELBO (18) over the kernel/prior hyperparameters while m_b, V_b, the rate parameters A, λ_0, and *the eigenbasis* B are held fixed. The free hyperparameters (all owned by the arc-cosine prior) are the six

$$\theta \in \{\ \sigma_0,\ \text{Amp},\ \beta,\ \rho,\ \varepsilon_{0x},\ \varepsilon_{0y}\ \}. \tag{44}$$

The eigenbasis B is held fixed as an approximation (for cost and stability) and re-synced at the start of the next outer iteration. This block is solved by gradient-based optimisation (e.g. LBFGS on the raw, unconstrained parameters); the hyperparameter gradients $\partial_\theta \mathcal{L}$ are obtained by automatic or analytical differentiation of the eigenspace ELBO.

Remark 6.3 (The amplitude Amp is optimized by default)**.** The amplitude prefactor Amp of the structured prior $C = \text{Amp}\,\alpha\, C_{\text{smooth}}\, \alpha^{\mathsf{T}}$ (Part I) is a genuine sixth kernel hyperparameter, not a fixed constant, and is optimized alongside the other five by default. The optional flag `fix_Amp` (default off) instead pins Amp = 1, recovering the amplitude-free model; left at its default, Amp trains with the rest.

Part III
Active learning: the information utility

7 The acquisition objective

The active-learning loop proceeds sequentially: after each count observation the model is refit, and the next input to query is the one whose observation is expected to teach us the most about the latent function. "Teach us the most" is made precise as an *information gain* — an expected reduction in the entropy of what we do not yet know about the latent function and its rate. This section builds that utility on the generative model and variational posterior of Parts I–II: the Poisson-lognormal predictive of the count, its entropy, and the two utilities considered here.

Throughout, the candidate (query) input is x^*; a count is the random variable R with realizations r (this is the count written y elsewhere in the document — we keep it as r here to match the count-domain notation $r_{\max}$ and the Laplace mode $\bar{g}_r$). The mean rate at input x is $f(x) = \exp(A\,\lambda(x) + \lambda_0)$ with latent $\lambda(x)$, gain A, and offset λ_0; the count is $R \mid f(x) \sim \text{Poisson}(f(x))$. It is convenient to work with the *log-rate* $g(x) = A\,\lambda(x) + \lambda_0$, so $f = e^g$, whose GP posterior at any input is Gaussian with the log-rate moments

$$\mu_g \;=\; A\,\mu(x^*) + \lambda_0, \qquad \sigma_g^2 \;=\; A^2\,\sigma^2(x^*), \tag{45}$$

the affine push-forward of the latent posterior moments $\mu(x^*)$, $\sigma^2(x^*)$ read off the variational posterior (16).

	Standard U_{std}	**Distribution-aware** U_{DA}
Formula	$H_{\text{marg}}(x^*) - \mathbb{E}[H_{\text{noise}}(x^*)]$	$H_{\text{marg}}(x^*) - \mathbb{E}_{x\sim p(x)}[H_{\text{cond}}(x^*)]$
As MI	$I\big(f(x^*); R \mid x^*, \mathcal{D}\big)$	$I\big(f(X); R \mid X, x^*, \mathcal{D}\big),\ X \sim p(x)$
Scores	rate *at* x^* *itself*	rates at *inputs* $x \sim p(x)$
Needs	marginal moments at x^*	marginal moments *and* cross-cov. to $p(x)$

Table 1: The two utilities. Both are "information gain = entropy reduction", about different targets.

Remark 7.1 (Two utilities, one idea)**.** We consider *two* acquisition functions. Both express the same idea — *utility*(x^*) = *information the observation* R *at* x^* *gives about the latent function = a drop in entropy* — and they differ only in *which* uncertainty they score (Table 1). The **standard utility** U_{std} scores the rate *at* x^* *itself*; the **distribution-aware utility** U_{DA} scores the rates at the inputs $x \sim p(x)$ we ultimately want to predict. The standard utility depends only on the marginal posterior at x^*; the distribution-aware utility couples x^* to the input distribution $p(x)$ through cross-covariance. U_{DA} is the richer object, but it carries a norm-exploitation divergence (§7.6, Thm. 8.9) that a finite candidate pool masks.

7.1 Utility as expected entropy reduction and mutual information

Fix a candidate x^* and the data $\mathcal{D} = \{(x_i, r_i)\}$. The utility is the mutual information between the count R we would observe at x^* and the latent function we want to learn. Precisely what we want to learn about is the difference between the two utilities.

The distribution-aware objective. Let $Z := (X, f(X))$ with $X \sim p(x)$ an input drawn from the input distribution and $f(X)$ its rate. Choosing x^* to maximize the information R carries about Z,

$$U_{\text{DA}}(x^*) \;:=\; I\big(Z; R \mid x^*, \mathcal{D}\big) \;=\; I\big((X, f(X));\, R \mid x^*, \mathcal{D}\big). \tag{46}$$

The chain rule for mutual information splits this into a term for *which* input we imagine and a term for its *rate*:

$$I\big((X, f(X)); R \mid x^*, \mathcal{D}\big) \;=\; \underbrace{I\big(X; R \mid x^*, \mathcal{D}\big)}_{=0} + I\big(f(X); R \mid X, x^*, \mathcal{D}\big). \tag{47}$$

The first term vanishes: *which* input X we intend to ask about is independent of the count R observed at x^*, given x^* and $\mathcal{D}$. Hence

$$U_{\text{DA}}(x^*) \;=\; I\big(f(X); R \mid X, x^*, \mathcal{D}\big) \;=\; \mathbb{E}_{x\sim p(x)}\big[\, I\big(f(x); R \mid x, x^*, \mathcal{D}\big)\big]. \tag{48}$$

Entropy decomposition. For a fixed input x, the per-input mutual information decomposes into a marginal-entropy term minus an expected conditional-entropy term,

$$I\big(f(x); R \mid x, x^*, \mathcal{D}\big) \;=\; H(R \mid x^*, \mathcal{D}) \;-\; \mathbb{E}_{f(x)}\big[\, H(R \mid x^*, f(x), \mathcal{D})\big], \tag{49}$$

using (i) $H(R \mid x, x^*, \mathcal{D}) = H(R \mid x^*, \mathcal{D})$ — the marginal of R at x^* does not depend on which input we intend to predict — and (ii) that conditioning on the latent value $f(x)$ leaves the expected entropy $\mathbb{E}_{f(x)}[H(R \mid x^*, f(x), \mathcal{D})]$. Averaging (49) over $x \sim p(x)$ and naming the two pieces gives the distribution-aware utility as a difference of entropies,

$$U_{\text{DA}}(x^*) \;=\; H_{\text{marg}}(x^*) \;-\; \mathbb{E}_{x\sim p(x)}\big[H_{\text{cond}}(x^*)\big], \tag{50}$$

with

$$H_{\text{marg}}(x^*) := H(R \mid x^*, \mathcal{D}), \tag{51}$$

$$H_{\text{cond}}(x^*) := \mathbb{E}_{x\sim p(x),\ \lambda(x)\sim q}\big[\, H(R \mid x^*, \lambda(x), \mathcal{D})\big]. \tag{52}$$

By the symmetry of mutual information, $U_{\text{DA}}(x^*)$ equals the reduction in uncertainty about the rates $\{f(x) : x \sim p(x)\}$ produced by observing R at x^* — the precise sense of "how much does querying x^* teach us about the latent function over the input distribution."

Remark 7.2 (Two expectations inside H_{cond})**.** H_{cond} in (52) carries *two* expectations: over inputs $x \sim p(x)$ and over the posterior latent $\lambda(x) \sim q$. The latent is integrated because $f(x) = \exp(A\lambda(x) + \lambda_0)$ is itself uncertain — only the posterior of $\lambda(x)$ is known, not its value. Replacing the inner expectation by an evaluation at the mean $\lambda(x) = \mu(x)$ is *biased*; the necessity of the inner sampling is the subject of §7.5.

The *standard* utility instead scores the rate at the query point itself. It is the special case in which the "input we predict" is x^*, so U_{DA}'s outer expectation collapses and H_{cond} becomes the aleatoric Poisson noise entropy at x^*; we develop it in §7.3 once H_{marg} is in hand.

7.2 The marginal count entropy H_{marg}

Both utilities share the marginal predictive entropy of the count at x^*,

$$H_{\text{marg}}(x^*) \;=\; -\sum_{r=0}^{\infty} p(r \mid x^*, \mathcal{D})\, \log p(r \mid x^*, \mathcal{D}), \tag{53}$$

where the marginal predictive integrates the Poisson likelihood against the Gaussian posterior of the log-rate $g(x^*) \sim \mathcal{N}(\mu_g, \sigma_g^2)$ (cf. (45)):

$$p(r \mid x^*, \mathcal{D}) \;=\; \int \text{Poisson}(r \mid e^g)\, \mathcal{N}(g \mid \mu_g, \sigma_g^2)\, dg \;=\; \frac{1}{r!}\int \exp(rg - e^g)\, \mathcal{N}(g \mid \mu_g, \sigma_g^2)\, dg. \tag{54}$$

This integral has no closed form. It is evaluated by a **Laplace expansion** of the integrand about its mode $\bar{g}_r$ (the mode of the log-integrand $rg - e^g - (g-\mu_g)^2/2\sigma_g^2$):

$$\log p(r \mid x^*, \mathcal{D}) \;\approx\; \bar{g}_r\, r - e^{\bar{g}_r} - \frac{(\bar{g}_r - \mu_g)^2}{2\sigma_g^2} - \tfrac{1}{2}\log(1 + \sigma_g^2\, e^{\bar{g}_r}) - \log r!. \tag{55}$$

Lemma 7.3 (Closed-form Laplace mode via Lambert W)**.** *The stationarity condition $r - e^g - (g-\mu_g)/\sigma_g^2 = 0$ rearranges to $g = r\sigma_g^2 + \mu_g - \sigma_g^2 e^g$. Substituting $w = \sigma_g^2 e^g$ gives $w + \log w = \log\sigma_g^2 + r\sigma_g^2 + \mu_g$, i.e. $w = W_0(\sigma_g^2\, e^{\,r\sigma_g^2+\mu_g})$ with W_0 the principal branch of the Lambert W function. Hence the mode is closed-form,*

$$\bar{g}_r \;=\; r\sigma_g^2 + \mu_g - W_0(\sigma_g^2\, e^{\,r\sigma_g^2+\mu_g}). \tag{56}$$

The infinite sum in (53) decays fast (rates are low) and is truncated at $r_{\max}$ (§7.6 handles the truncation artifact). At very small variance the Laplace form is unstable, and one falls back to the exact Poisson log-pmf $\log p(r) = r\mu_g - e^{\mu_g} - \log r!$ (used for $\sigma_g^2 < 10^{-6}$). The mode is computed in log-space — $W_0(e^y)$ with $y = \log\sigma_g^2 + r\sigma_g^2 + \mu_g$, Newton-iterated with the derivative $dW/dy = W/(1+W)$ — so no $e^{r\sigma_g^2+\mu_g}$ is ever formed (overflow-safe and differentiable in x^*).

7.3 The standard utility

The standard acquisition is the mutual information between the count and the latent rate *at the query point itself*,

$$U_{\text{std}}(x^*) \;=\; H_{\text{marg}}(x^*) - \mathbb{E}[H_{\text{noise}}(x^*)] \;=\; H(R \mid x^*, \mathcal{D}) - \mathbb{E}_{f\sim\text{post.}}[H(R \mid f)] \;=\; I(f(x^*); R \mid x^*, \mathcal{D}). \tag{57}$$

It scores "how much would a count at x^* pin down the rate at x^*" — it depends on the marginal GP moments at x^* only, with no $p(x)$ and no cross-covariance. It is exactly U_{DA} with the predicted input taken to be the query point, so the second term is the *aleatoric* Poisson noise entropy rather than an epistemic conditional entropy.

7.3.1 The aleatoric noise entropy $\mathbb{E}[H_{\text{noise}}]$ in closed form

Given a rate f, the entropy of Poisson(f) is $H_{\text{noise}}(f) = f\,(1 - \log f) + \mathbb{E}_{r\sim\text{Poisson}(f)}[\log r!]$. Writing $g = \log f \sim \mathcal{N}(\mu_g, \sigma_g^2)$ and using the lognormal moment identities

$$\mathbb{E}[e^g] = \exp\!\big(\mu_g + \tfrac{1}{2}\sigma_g^2\big) = \bar{f}(x^*), \qquad \mathbb{E}[g\,e^g] = \exp\!\big(\mu_g + \tfrac{1}{2}\sigma_g^2\big)\,(\mu_g + \sigma_g^2), \tag{58}$$

(the first is the expected rate $\bar{f}$ of (21)), the first piece integrates in closed form:

$$\mathbb{E}_g\big[f(1-\log f)\big] = \mathbb{E}_g\big[e^g(1-g)\big] = \exp\!\big(\mu_g+\tfrac{1}{2}\sigma_g^2\big)\big(1-(\mu_g+\sigma_g^2)\big) = -\exp\!\big(\mu_g+\tfrac{1}{2}\sigma_g^2\big)(\mu_g+\sigma_g^2-1). \tag{59}$$

The remaining $\mathbb{E}[\log r!]$ is taken under the marginal predictive $p(r \mid x^*, \mathcal{D})$ of (54). Hence

$$\mathbb{E}[H_{\text{noise}}(x^*)] \;=\; -\exp\!\big(\mu_g + \tfrac{1}{2}\sigma_g^2\big)(\mu_g + \sigma_g^2 - 1) \;+\; \sum_r p(r \mid x^*, \mathcal{D})\,\log r!. \tag{60}$$

This is the closed form for $\mathbb{E}[H_{\text{noise}}]$. Assembling U_{std} then reads the marginal latent moments $\mu(x^*), \sigma^2(x^*)$, transforms them to μ_g, σ_g^2 by (45), and returns $U_{\text{std}} = H_{\text{marg}} - \mathbb{E}[H_{\text{noise}}]$.

7.4 The distribution-aware utility

The distribution-aware utility (50) reduces uncertainty about the rates at the inputs we actually want to predict. It weights a candidate x^* by how informative its observation is about the rates at typical inputs — i.e. by the cross-covariance between x^* and inputs $x \sim p(x)$ ("distribution-aware" = aware of the input distribution $p(x)$). Evaluating H_{cond} needs the predictive moments of $\lambda(x^*)$ *after a hypothetical observation* $\lambda(x) = \lambda_i$ at an input x.

7.4.1 Conditional GP moments via Gaussian conditioning

Let $u(x) = \widetilde{K}^{-1}k(x)$ be the SVGP projection and $q(\widetilde{\lambda}) = \mathcal{N}(m, V)$ the variational posterior on the inducing values. The marginal latent posterior moments at x and x^* are the projected moments of (16), $\mu(x) = u(x)^{\mathsf{T}}m$, $\sigma^2(x) = s(x) + u(x)^{\mathsf{T}}Vu(x)$ (and likewise at x^*), where $s(x) = k(x,x) - u(x)^{\mathsf{T}}k(x)$ is the prior Schur complement. The object that couples the two points is the **prior conditional covariance** of $\lambda(x), \lambda(x^*)$ given the inducing values:

$$c \;=\; k(x, x^*) - u(x)^{\mathsf{T}}\widetilde{K}\,u(x^*). \tag{61}$$

By the law of total covariance the full posterior cross-covariance is

$$\Sigma_{x^*} \;=\; c + u(x)^{\mathsf{T}}V\,u(x^*) \;=\; k(x, x^*) + u(x)^{\mathsf{T}}(V - \widetilde{K})\,u(x^*), \tag{62}$$

and standard Gaussian conditioning on the joint of $(\lambda(x), \lambda(x^*))$ gives the conditional latent moments at x^* after observing $\lambda(x) = \lambda_i$,

$$\mu_{\text{cond}}(x^*) = \mu(x^*) + \frac{\Sigma_{x^*}}{\sigma^2(x)}\,(\lambda_i - \mu(x)), \tag{63}$$

$$\sigma^2_{\text{cond}}(x^*) = \sigma^2(x^*) - \frac{\Sigma^2_{x^*}}{\sigma^2(x)}. \tag{64}$$

These are transformed to log-rate space, $A\,\mu_{\text{cond}} + \lambda_0$ and $A^2\sigma^2_{\text{cond}}$, and fed to the same entropy functional (53) to give the per-input conditional entropy $H(R \mid x^*, \lambda(x), \mathcal{D})$. The full Gaussian-conditioning derivation (augmented inducing matrix, block inverse, and the equivalence to the rank-1 posterior update) is the deep-layer result (72); and the conditional mean (63) obtained by conditioning on the joint posterior coincides with the mean obtained by augmenting the inducing set with the hypothetical observation and re-integrating (Thm. 8.5). Neither is re-derived here.

Remark 7.4 (The cross-covariance pitfall: c must be included)**.** Forming the joint over $[x; x^*]$ and reading the *full* posterior covariance between the two points already yields $c + u^\mathsf{T} V u^* = \Sigma_{x^*}$ — the c term of (61) included. The pitfall is to instead assemble the $u^\mathsf{T} V u^*$-only cross-covariance by hand, dropping c: this is biased outside the inducing hull. Taking $\mu_{\text{cond}}, \sigma^2_{\text{cond}}$ from the blocks of the complete joint covariance (with a floor $\sigma^2_{\text{cond}} \geq 10^{-8}$) avoids it.

Monte-Carlo estimator (one λ per input). Given pre-drawn samples $\{x_i\}_{i=1}^{N_{\text{mc}}} \sim p(x)$:

1. $H_{\text{marg}}(x^*)$ = entropy (53) at the marginal moments μ_g, σ^2_g of x^*.
2. For $i = 1, \ldots, N_{\text{mc}}$: read $(\mu(x_i), \sigma^2(x_i))$; draw $\lambda_i = \mu(x_i) + \sqrt{\sigma^2(x_i)}\,\varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$ (sampled variant; the deterministic variant sets $\lambda_i = \mu(x_i)$); form the conditional moments (63)–(64) at x^*; set $H^{(i)}_{\text{cond}}$ = entropy at $(A\mu_{\text{cond}} + \lambda_0,\ A^2\sigma^2_{\text{cond}})$.
3. $H_{\text{cond}} = \frac{1}{N_{\text{mc}}} \sum_i H^{(i)}_{\text{cond}}$; $\quad U_{\text{DA}} = H_{\text{marg}} - H_{\text{cond}}$.

The input posteriors and the reparameterized draws λ_i do not depend on x^*, so they are held fixed (detached from the gradient) when differentiating in x^*, avoiding $O(N_{\text{mc}})$ graph growth; the conditional moments at x^* remain differentiable in x^*. The two variants: the deterministic one takes $\lambda_i = \mu(x_i)$ (a reproducible sanity check — §7.4.2); the sampled one is the correct unbiased estimator (§7.5).

7.4.2 The deterministic special case

Setting $\lambda_i = \mu(x)$ (the posterior mean) makes the innovation $\lambda_i - \mu(x) = 0$, so (63)–(64) collapse to $\mu_{\text{cond}} = \mu(x^*)$ (mean unchanged) and

$$\sigma^2_{\text{cond}}(x^*) \;=\; \sigma^2(x^*)\,(1 - \rho^2_\lambda), \qquad \rho^2_\lambda \;=\; \frac{\Sigma^2_{x^*}}{\sigma^2(x)\,\sigma^2(x^*)}, \tag{65}$$

where ρ_λ is the posterior latent correlation between $\lambda(x)$ and $\lambda(x^*)$. In log-rate coordinates the conditioning map is $(\mu_g, \sigma^2_g) \mapsto (\mu_g, \sigma^2_g(1 - \rho^2_\lambda))$: same mean, variance scaled by $1 - \rho^2_\lambda$. Writing $H_{\text{marg}}(\cdot, \cdot)$ for the count entropy (53) as a function of the log-rate moments, the utility is the entropy difference

$$U_{\text{DA}}(x^*) \;=\; H_{\text{marg}}(\mu_g, \sigma^2_g) \;-\; H_{\text{marg}}\big(\mu_g,\ \sigma^2_g(1 - \rho^2_\lambda)\big). \tag{66}$$

So the deterministic U_{DA} depends on exactly three quantities: the operating point μ_g, the marginal log-rate variance σ_g^2 (how much entropy to start with), and ρ_λ^2 (the fraction of that variance conditioning removes). ρ_λ^2 is governed by the angle between x^* and the conditioning input in C-space: a small angle gives $\rho_\lambda^2 \to 1$ (near-total variance removal), a large angle $\rho_\lambda^2 \to 0$ (no conditioning). This same ρ_λ^2 sets the sampling bias of §7.5.

7.5 Why we sample the latent instead of using its mean

H_{cond} is a double expectation (Remark 7.2), over $x \sim p(x)$ and over $\lambda(x) \sim q$. The tempting shortcut replaces the inner expectation by evaluation at the mean,

$$\widetilde{H}_{\mathrm{cond}}(x^*) \;=\; \mathbb{E}_{x\sim p(x)}\big[\, H(R \mid x^*, \lambda(x) = \mu(x), \mathcal{D})\big]. \tag{67}$$

Proposition 7.5 (The mean shortcut is biased)**.** $\mathbb{E}_{\lambda(x)}[H(R \mid x^*, \lambda(x), \mathcal{D})] \neq H(R \mid x^*, \lambda(x) = \mu(x), \mathcal{D})$ *in general. The predictive mean of* $\lambda(x^*)$ *is linear in the conditioning value,* $\mathbb{E}[\lambda(x^*) \mid \lambda(x), \mathcal{D}] = \mu(x^*) + \alpha\,(\lambda(x) - \mu(x))$ *with regression coefficient* $\alpha = \mathrm{Cov}(\lambda(x^*), \lambda(x) \mid \mathcal{D})/\sigma^2(x)$*, while the predictive* variance *does not depend on the value of* $\lambda(x)$*. Entropy is nonlinear in the predictive mean, so a second-order Taylor expansion of* H *about* $\lambda(x) = \mu(x)$*, with* $\mathbb{E}[\lambda(x) - \mu(x)] = 0$*, leaves the leading bias*

$$\mathbb{E}_{\lambda(x)}[H] - H\big|_{\lambda(x)=\mu(x)} \;\approx\; \tfrac{1}{2}\,\alpha^2\,\sigma^2(x)\,\frac{\partial^2 H_{\mathrm{marg}}}{\partial \mu^2}\Big|_{x^*}. \tag{68}$$

Since $\mathrm{Cov}(\lambda(x^*), \lambda(x)) = \rho_\lambda \sqrt{\sigma^2(x^*)\sigma^2(x)}$*, we have* $\alpha^2\,\sigma^2(x) = \rho_\lambda^2\,\sigma^2(x^*)$*, and the* $\sigma^2(x)$ *cancels:*

$$\mathrm{Bias} \;\approx\; \frac{\rho_\lambda^2}{2}\,\sigma^2(x^*)\,\frac{\partial^2 H_{\mathrm{marg}}}{\partial \mu^2}\Big|_{x^*}. \tag{69}$$

The bias vanishes *only* when $\rho_\lambda = 0$ (x uninformative about x^* — but then the utility contribution is ≈ 0 anyway), or $\sigma^2(x^*) = 0$ (nothing left to learn), or $\partial^2 H_{\mathrm{marg}}/\partial\mu^2 = 0$ (entropy linear in the mean — false for the Poisson-lognormal). Notably $\sigma^2(x) = 0$ is *not* a zero-bias condition: it cancels in (69).

Remark 7.6 (The bias is largest exactly where it matters)**.** Bias $\propto \rho_\lambda^2\,\sigma^2(x^*)$ is maximal when the sampled input x is most correlated with x^* (most relevant to the utility) *and* when x^* is most uncertain (where the acquisition should be steering us) — i.e. largest as $\rho_\lambda \to 1$. A systematic bias does not shrink with more Monte-Carlo samples: it converges to the *wrong* value and corrupts the optimizer's direction. Sampling λ is unbiased and its variance falls as $O(1/N)$. Hence: sample λ.

One λ per input is optimal (nested MC). With $g(x, \lambda) = H(R \mid x^*, \lambda, \mathcal{D})$, let $\tau^2 = \mathrm{Var}_x[\mathbb{E}_\lambda g]$ (between-input) and $\mathbb{E}_x[\sigma_x^2]$ (within-input). For a fixed budget of M entropy evaluations, using N_λ latent samples per input gives variance $(N_\lambda \tau^2 + \mathbb{E}_x[\sigma_x^2])/M$, strictly worse for $N_\lambda > 1$. Spending the budget on *more inputs*, one λ each, is optimal.

Caveat. The magnitude of this bias is a provisional estimate. Pushed to log-rate space the curvature carries a gain factor, $\partial^2 H_{\mathrm{marg}}/\partial\mu^2 = A^2\,\partial^2 H_{\mathrm{marg}}/\partial g^2$, so Bias $\propto A^2$; for a small gain (e.g. $A \approx 0.03$) the bias is far below the MC standard error, and the deterministic and sampled U_{DA} nearly coincide. The regime guide $A \ll 1$ negligible / $A \sim 1$ moderate / $A \gg 1$ essential, and the specific numbers, are plausibility arguments, not settled results. The core claim — bias (69) does not vanish with N — is solid; the magnitude estimate is provisional. The *correct* estimator remains the sampled one regardless.

7.6 The entropy landscape over input space

Monotone shape — utility is not pure epistemic uncertainty. The count entropy $H_{\text{marg}}(\mu_g, \sigma_g^2)$ increases monotonically in *both* arguments: with μ_g (higher predicted rate) and with σ_g^2 (more GP/epistemic uncertainty). A candidate therefore scores high for a *mixture* of reasons — it is predicted to have a high rate *and* it is epistemically uncertain. The utility is thus not a pure epistemic-uncertainty acquisition; predicted rate is baked in. This is a deliberate modelling choice: a normalized arc-cosine kernel would force $K(x, x) = 1$, removing the norm channel so that utility tracked only angular alignment (pure epistemic), but the held-out predictive correlation then drops by $\sim$25% (a fitted example, $M = 100$: $0.79 \to 0.59$) — input norm is genuinely informative, so the kernel keeps the norm dependence.

The $r_{\max}$ truncation artifact. With a fixed $r_{\max} = 100$ the Laplace sum (53) misses the probability mass above $r_{\max}$ once the Poisson peak climbs past it, and H_{marg} *collapses to* ≈ 0 for $\mu_g \gtrsim \log 100 \approx 4.6$ (and at lower μ_g for large σ_g^2). This is a numerical artifact, not real entropy: the computable region is roughly triangular in (μ_g, σ_g^2). A cheap pre-check without running the Laplace sum is $z_{\text{safe}} = (\log r_{\max} - \mu_g)/\sqrt{\sigma_g^2}$ — $z_{\text{safe}} > 2$ safe, < 1.5 unreliable, < 0.5 broken. Inputs from the pool sit at $z_{\text{safe}} \gg 10$, always safe; trouble arises only for artificially amplified inputs. An adaptive choice of $r_{\max}$, sized from the moments so the truncation always covers the Poisson mass, removes the collapse.

Norm-exploitation divergence. Because the arc-cosine kernel is positively 1-homogeneous ($K(\kappa x, y) = \kappa\, K(x, y)$ when the bias $\sigma_0 = 0$), scaling $x^* \to \kappa x^*$ scales $\mu(x^*) \sim \kappa$ and $\sigma^2(x^*) \sim \kappa^2$, so H_{marg} grows while a directionally aligned conditioning input keeps $\rho_\lambda \approx 1$ and $\sigma^2_{\text{cond}} \approx 0$. Under free-space gradient ascent U_{DA} then grows $\sim \kappa^{1.9}$ (numerically) — the optimizer "diverges" by amplifying input norm rather than finding angularly informative inputs. This is intrinsic to the homogeneous kernel, *not* a bug; the full statement and the $\sigma_0^2 > 0$ damping (alignment peak strictly at $\kappa = 1$) are Thm. 8.9 in the deep layer. Over a finite *pool* of candidate inputs (discrete selection) norms are bounded and this never triggers.

Caveat. Utility accuracy ceiling (known limitation). At high rate the two utilities fail in opposite directions through the same $r_{\max} = 100$ truncation: U_{std} can run to $+\infty$ while U_{DA} silently collapses toward 0. A rate guard ($f_{\max}$, default 100) keeps the operating point in the accurate regime; a dedicated look-up table for the U_{DA} path was not built. This is a limitation of the fixed-$r_{\max}$ path, cured on the standard path by the adaptive choice of $r_{\max}$.

8 The deep layer: conditioning, subspace, divergence, and numerics

Part III (§7) *defined* the two acquisition utilities and their entropies; this section proves *why* those formulas behave as they do. It supplies the theorems underlying the moment formulas, the exact predictive-conditioning derivation, the subspace-validity argument, the norm-scaling divergence theorems that explain the utility's blow-up under unconstrained input optimization, the two kernel-level fixes, and the four-failure-mode numerical audit of the standard utility.

Two objects recur, and their "subtracted term" is *not* the same (the single most load-bearing clash; the top-level exposition is in Part III):

- the **standard** utility $U_{\text{std}}(x^*) = H_{\text{marg}}(x^*) - \mathbb{E}_g[H_{\text{noise}}(x^*)]$ of (57), which the selection loop

maximizes (§8.6 audits it); its subtracted term is the expected Poisson *noise* entropy $\mathbb{E}_g[H_{\text{noise}}]$, an average over the point's *own* latent, no second input involved;

- the **distribution-aware (DA)** utility $U_{\text{DA}}(x^*) = H_{\text{marg}}(x^*) - \mathbb{E}_{x\sim p(x)}[H_{\text{cond}}(x^*)]$ of (50), whose subtracted term is the entropy *after Gaussian-conditioning* the GP on a hypothetical observation of $\lambda(x_t)$ at a target input x_t.

The two coincide only in the $\rho_\lambda^2 = 1$ deterministic limit, where H_{cond} collapses to a single Poisson entropy (§8.4). The divergence theorems of §8.4 and the remedies of §8.5 concern *gradient ascent on the input* x^* (a free input-synthesis setting), which is distinct from selection over a fixed pool.

8.1 GP moments and Gaussian conditioning (Proof III)

This subsection derives the marginal and conditional GP moments underlying the utilities of Part III.

8.1.1 The self-kernel identity

Recall the order-1 arc-cosine kernel of Part I, (2),

$$K(x,x') \;=\; \tfrac{1}{\pi}\,\mathcal{M}\,J(\theta), \qquad \mathcal{M} = \sqrt{v_x\, v_{x'}}, \quad \cos\theta = \frac{x^\mathsf{T} C x' + \sigma_0^2}{\mathcal{M}}, \quad J(\theta) = \sin\theta + (\pi-\theta)\cos\theta,$$

with self-magnitude $v_x = x^\mathsf{T} C x + \sigma_0^2$.

Lemma 8.1 (Self-kernel / prior variance)**.** *For every x, $K(x,x) = \|x\|_C^2 + \sigma_0^2 = v_x$, where $\|x\|_C^2 \equiv x^\mathsf{T} C x$.*

Proof. Set $x' = x$: $\cos\theta = v_x/v_x = 1 \Rightarrow \theta = 0$, and $J(0) = \sin 0 + \pi\cos 0 = \pi$. Hence $K(x,x) = \tfrac{1}{\pi}\, v_x\, \pi = v_x = x^\mathsf{T} C x + \sigma_0^2$. □

This restates (5): *the prior variance at any point equals its squared C-norm (plus the bias variance).* It is the single identity from which the entire norm-scaling divergence of §8.4 grows.

8.1.2 Variational posterior moments

With M inducing points $\{\widetilde{z}_j\}$ and variational posterior $q(\widetilde{\lambda}) = \mathcal{N}(m, V)$, write the inducing gram $\widetilde{K} = K(\widetilde{Z},\widetilde{Z})$, the cross-kernel vector $k(x) = [K(x,\widetilde{z}_1),\dots,K(x,\widetilde{z}_M)]^\mathsf{T}$, the projection $u(x) = \widetilde{K}^{-1}k(x)$, and the (training-fixed) correction matrix

$$W \;=\; \widetilde{K}^{-1}(V - \widetilde{K})\,\widetilde{K}^{-1}.$$

The marginal moments are

$$\begin{aligned} \mu(x^*) &= k(x^*)^\mathsf{T}\widetilde{K}^{-1} m, \\ \sigma^2(x^*) &= K(x^*,x^*) + k(x^*)^\mathsf{T} W\, k(x^*) \\ &= \|x^*\|_C^2 + \sigma_0^2 + k(x^*)^\mathsf{T} W\, k(x^*). \end{aligned} \tag{70}$$

The first term of σ^2 is the prior variance (Lemma 8.1); the second is the posterior correction (negative when $V \prec \widetilde{K}$, i.e. data shrink variance). Between two points,

$$\Sigma_q(x^*,x_t) = K(x^*,x_t) + k(x^*)^\mathsf{T} W\, k(x_t), \qquad \Sigma_q(x^*,x^*) = \sigma^2(x^*), \quad \Sigma_q(x_t,x_t) = \sigma^2(x_t),$$

and the squared posterior correlation — the Pearson correlation of the joint Gaussian $[\lambda(x^*), \lambda(x_t)]$ under q — is

$$\rho^2_\lambda(x^*, x_t) = \frac{\Sigma_q(x^*, x_t)^2}{\sigma^2(x^*)\,\sigma^2(x_t)} \in [0, 1]. \tag{71}$$

8.1.3 Conditioning on an observed latent

Observing λ_t at x_t and applying the standard Gaussian conditioning identity to $[\lambda(x^*), \lambda(x_t)]$ gives the conditional moments:

$$\boxed{\begin{aligned} \mu_{\text{cond}}(x^*) &= \mu(x^*) + \frac{\Sigma_q(x^*, x_t)}{\sigma^2(x_t)}(\lambda_t - \mu(x_t)), \\ \sigma^2_{\text{cond}}(x^*) &= \sigma^2(x^*) - \frac{\Sigma_q(x^*, x_t)^2}{\sigma^2(x_t)} = \sigma^2(x^*)(1 - \rho^2_\lambda). \end{aligned}} \tag{72}$$

Proposition 8.2 (ρ^2_λ is the fractional variance reduction)**.** *The conditional variance at x^* is the marginal variance scaled by $(1 - \rho^2_\lambda)$; equivalently, conditioning on $\lambda(x_t)$ removes a fraction ρ^2_λ of the variance at x^*.*

Proof. Substitute the correlation definition (71) into the variance line of (72):

$$\sigma^2(x^*) - \frac{\Sigma_q(x^*, x_t)^2}{\sigma^2(x_t)} = \sigma^2(x^*)\Big[1 - \frac{\Sigma_q(x^*, x_t)^2}{\sigma^2(x^*)\,\sigma^2(x_t)}\Big] = \sigma^2(x^*)(1 - \rho^2_\lambda).$$

□

The conditional moments are obtained by conditioning on the *full joint* covariance of $[x_t; x^*]$ and applying exactly (72) (with a floor $\sigma^2_{\text{cond}} \geq 10^{-8}$). By the theorem of §8.2 this joint route is the *exact* augmented predictive, not the sparse approximation.

8.1.4 Log-rate transform and the DA utility in three scalars

Under $g = A\lambda + \lambda_0$ and $R \sim \text{Poisson}(e^g)$, the GP moments push to g-space as $\mu_g = A\mu + \lambda_0$, $\sigma^2_g = A^2\sigma^2$ (marginal) and $\mu_{g,\text{cond}} = A\,\mu_{\text{cond}} + \lambda_0$, $\sigma^2_{g,\text{cond}} = A^2\sigma^2_{\text{cond}} = \sigma^2_g(1 - \rho^2_\lambda)$ (the variance reduction carries through). Hence, writing $H_{\text{marg}}(\mu_g, \sigma^2_g)$ for the marginal Poisson–log-normal entropy (53),

$$U_{\text{DA}}(x^*) = H_{\text{marg}}(\mu_g, \sigma^2_g) - H_{\text{marg}}\Big(\mu_{g,\text{cond}},\, \sigma^2_{g,\text{cond}}\Big)\,.$$

In deterministic mode ($\lambda_t = \mu(x_t)$) the conditional mean is unchanged, $\mu_{g,\text{cond}} = \mu_g$, and (50) reduces to the deterministic distribution-aware utility (66) of §7.4.2, a function of exactly three scalars:

$$\boxed{U_{\text{DA}}(x^*) = H_{\text{marg}}(\mu_g, \sigma^2_g) - H_{\text{marg}}\Big(\mu_g,\ \sigma^2_g(1 - \rho^2_\lambda)\Big)} \tag{73}$$

the operating point $\mu_g = A\mu(x^*) + \lambda_0$, the marginal variance $\sigma^2_g = A^2\sigma^2(x^*)$, and the fractional reduction ρ^2_λ. Conditioning slides $(\mu_g, \sigma^2_g) \to (\mu_g, \sigma^2_g(1 - \rho^2_\lambda))$: same mean, reduced variance; the utility is the entropy gap. This is the object whose divergence §8.4 analyses.

8.2 The predictive distribution conditioned on a new observation

The DA utility integrates the entropy of $p(\lambda(x^*) \mid \lambda(x), \mathcal{D})$: the predictive over the latent at a query x^* after a *hypothetical* new latent observation at x. We derive it two ways — sparse-approximated and exact/augmented — and prove the two agree in mean.

Define the *prior conditional covariance* (the residual not captured by the inducing points)

$$c(x, x') \;\equiv\; K(x, x') - u(x)^\mathsf{T} \widetilde{K}\, u(x') \;=\; K(x, x') - k(x)^\mathsf{T} \widetilde{K}^{-1} k(x'),$$

whose diagonal is the residual / aleatoric variance $\sigma_r^2(x) \equiv c(x, x) = K(x, x) - k(x)^\mathsf{T} \widetilde{K}^{-1} k(x) \geq 0$.

8.2.1 The rank-1 posterior update

Lemma 8.3 (Rank-1 update)**.** *Let the new point x have projection $u = u(x)$ and residual variance $\sigma_r^2 = \sigma_r^2(x)$, so the conditional likelihood is $p(\lambda(x) \mid \widetilde{\lambda}) = \mathcal{N}(u^\mathsf{T}\widetilde{\lambda}, \sigma_r^2)$. Conditioning the variational posterior $\mathcal{N}(m, V)$ on the observed $\lambda(x)$ yields $\mathcal{N}(m', V')$ with*

$$m' \;=\; m + \frac{Vu}{\sigma_r^2 + u^\mathsf{T} V u}(\lambda(x) - u^\mathsf{T} m), \qquad V' \;=\; V - \frac{Vu\, u^\mathsf{T} V}{\sigma_r^2 + u^\mathsf{T} V u}. \tag{74}$$

Proof. Under the variational approximation the joint of $(\lambda(x), \widetilde{\lambda})$ is Gaussian:

$$\begin{bmatrix} \lambda(x) \\ \widetilde{\lambda} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} u^\mathsf{T} m \\ m \end{bmatrix}, \begin{bmatrix} \sigma_r^2 + u^\mathsf{T} V u & u^\mathsf{T} V \\ Vu & V \end{bmatrix} \right).$$

The Gaussian conditioning identity on $\widetilde{\lambda}$ given $\lambda(x)$, with scalar lower-right-to-upper-left Schur term $\Sigma_{11} = \sigma_r^2 + u^\mathsf{T} V u$ and cross-block Vu, gives (74) directly. □

8.2.2 Sparse vs. augmented predictive

Let $u^* = \widetilde{K}^{-1} k(x^*)$ and $\sigma_r^2(x^*) = K(x^*, x^*) - k(x^*)^\mathsf{T} \widetilde{K}^{-1} k(x^*)$.

Sparse route. Assuming conditional independence $p(\lambda(x^*) \mid \lambda(x), \widetilde{\lambda}) \approx p(\lambda(x^*) \mid \widetilde{\lambda})$ and integrating the conditional prior against the updated posterior of Lemma 8.3,

$$\mu_{\text{pred}} = {u^*}^\mathsf{T} m', \qquad \sigma^2_{\text{pred}} = \sigma_r^2(x^*) + {u^*}^\mathsf{T} V' u^* = K(x^*, x^*) - {u^*}^\mathsf{T}(\widetilde{K} - V') u^*.$$

Augmented (exact) route. Keep the direct $\lambda(x)$–$\lambda(x^*)$ correlation. With augmented latent $\widetilde{\lambda}_{\text{aug}} = [\lambda(x); \widetilde{\lambda}]$ and augmented gram / cross-kernel

$$\widetilde{K}_x = \begin{bmatrix} K(x, x) & k(x)^\mathsf{T} \\ k(x) & \widetilde{K} \end{bmatrix}, \qquad k_{\text{aug}}(x^*) = \begin{bmatrix} K(x, x^*) \\ k(x^*) \end{bmatrix},$$

the non-approximated conditional prior is $p(\lambda(x^*) \mid \widetilde{\lambda}_{\text{aug}}) = \mathcal{N}(u_{\text{aug}}^\mathsf{T} \widetilde{\lambda}_{\text{aug}}, S^*)$, $u_{\text{aug}} = \widetilde{K}_x^{-1} k_{\text{aug}}(x^*)$, $S^* = K(x^*, x^*) - u_{\text{aug}}^\mathsf{T} \widetilde{K}_x\, u_{\text{aug}}$. Since $\lambda(x)$ is *observed* (variance 0) while $\widetilde{\lambda}$ follows the updated posterior $\mathcal{N}(m', V')$, the augmented posterior covariance is $V'' = \left[\begin{smallmatrix} 0 & 0^\mathsf{T} \\ 0 & V' \end{smallmatrix}\right]$, and

$$\mu_{\text{pred}} = u_{\text{aug}}^\mathsf{T}[\lambda(x); m'], \qquad \sigma^2_{\text{pred}} = S^* + u_{\text{aug}}^\mathsf{T} V'' u_{\text{aug}} = K(x^*, x^*) - u_{\text{aug}}^\mathsf{T}(\widetilde{K}_x - V'')\, u_{\text{aug}}.$$

Remark 8.4 (Augmented route fixes the sparse self-variance artifact)**.** When $x = x^*$, u_{aug} aligns with the first column of $\widetilde{K}_x$ and, because the $(1, 1)$ block of V'' is 0, the predictive variance is forced to 0. A self-consistent predictive *must* have zero variance at an already-observed point; the sparse route leaves a spurious nonzero self-variance there. This is the same pathology as a separately-assembled cross-covariance that can violate Cauchy–Schwarz (Part II §4.7); conditioning on the full joint covariance sidesteps both.

8.2.3 Equivalence of predictive means

Theorem 8.5 (Equivalence of predictive means)**.** *The augmented-integration predictive mean equals the mean obtained by standard Gaussian conditioning on the joint predictive* $[\lambda(x), \lambda(x^*)]$ *— exactly, with no cross-covariance approximation.*

Proof. **Method 1 (joint conditioning).** The joint moments given $\mathcal{D}$ are $\mu_x = u^\mathsf{T} m$, $\mu_{x^*} = u^{*\mathsf{T}} m$,

$$\Sigma_{11} = \sigma_r^2 + u^\mathsf{T} V u, \qquad \Sigma_{12} = \underbrace{\big(K(x,x^*) - u^\mathsf{T} \widetilde{K}\, u^*\big)}_{=\,c} + u^{*\mathsf{T}} V u,$$

where the residual cross term is exactly $c = c(x, x^*)$. The Gaussian conditioning identity gives the target

$$\mathbb{E}[\lambda(x^*) \mid \lambda(x), \mathcal{D}] = \mu_{x^*} + \frac{\Sigma_{12}}{\Sigma_{11}}(\lambda(x) - \mu_x) = u^{*\mathsf{T}} m + \frac{\Sigma_{12}}{\Sigma_{11}}(\lambda(x) - u^\mathsf{T} m). \tag{75}$$

Method 2 (augmented integration). Block-invert $\widetilde{K}_x$. Writing the residual-correlation coefficient $\vartheta \equiv c/\sigma_r^2$ (denoted ϑ rather than α to avoid clashing with the localization weight α_i),

$$u_{\text{aug}} = \begin{bmatrix} \vartheta \\ u^* - \vartheta\, u \end{bmatrix}, \qquad \vartheta = \frac{K(x,x^*) - u^\mathsf{T} \widetilde{K}\, u^*}{\sigma_r^2} = \frac{c}{\sigma_r^2}.$$

Then, with m' from (74),

$$\mu_{\text{aug}} = u_{\text{aug}}^\mathsf{T}[\lambda(x); m'] = \vartheta\, \lambda(x) + (u^{*\mathsf{T}} - \vartheta\, u^\mathsf{T}) m' = \vartheta\, \lambda(x) + (u^{*\mathsf{T}} - \vartheta\, u^\mathsf{T}) \left[m + \frac{Vu}{\Sigma_{11}}(\lambda(x) - u^\mathsf{T} m) \right].$$

Group by the prior mean $u^{*\mathsf{T}} m$ and the innovation $(\lambda(x) - u^\mathsf{T} m)$, adding and subtracting $\vartheta\, u^\mathsf{T} m$:

$$\mu_{\text{aug}} = u^{*\mathsf{T}} m + \underbrace{\left[\vartheta + \frac{u^{*\mathsf{T}} V u - \vartheta\, u^\mathsf{T} V u}{\Sigma_{11}} \right]}_{\mathcal{C}} (\lambda(x) - u^\mathsf{T} m).$$

Put $\mathcal{C}$ over the common denominator $\Sigma_{11} = \sigma_r^2 + u^\mathsf{T} V u$ and use $\vartheta\, \sigma_r^2 = c$:

$$\mathcal{C} = \frac{\vartheta\, \sigma_r^2 + \vartheta\, u^\mathsf{T} V u + u^{*\mathsf{T}} V u - \vartheta\, u^\mathsf{T} V u}{\Sigma_{11}} = \frac{\vartheta\, \sigma_r^2 + u^{*\mathsf{T}} V u}{\Sigma_{11}} = \frac{c + u^{*\mathsf{T}} V u}{\Sigma_{11}} = \frac{\Sigma_{12}}{\Sigma_{11}}.$$

Hence $\mu_{\text{aug}} = u^{*\mathsf{T}} m + (\Sigma_{12}/\Sigma_{11})(\lambda(x) - u^\mathsf{T} m)$, identical to (75). □

Remark 8.6. Theorem 8.5 certifies that conditioning on the full joint covariance (§8.1) computes the *exact* augmented predictive mean — it does not incur the sparse cross-covariance error. Setting the residual cross term $c = 0$ would enforce the sparse factorization

$$p(\lambda^*, \lambda \mid \widetilde{\lambda}) \approx p(\lambda^* \mid \widetilde{\lambda})\, p(\lambda \mid \widetilde{\lambda});$$

the inducing (epistemic) cross-covariance $u^{*\mathsf{T}} V u$ can still be nonzero, because changing belief about an inducing point moves both $\lambda(x^*)$ and $\lambda(x)$.

8.3 Subspace theory

Why the utility may be optimized in a low-dimensional subspace of the input space ($d = 11664$ grid points in full; ∼900–2356 in the localized-support region) instead of the full input space. Setting: optimize x^* to maximize $U_{\text{DA}}(x^*)$ by gradient ascent, constrained to $x_{\text{loc}} = \bar{x} + (\text{basis}) \cdot \zeta$ over the subspace coordinate ζ; the offset $\bar{x}$ is the dataset mean over the localized-support coordinates, so $\zeta = 0$ maps to the mean input.

8.3.1 The kernel touches x only through the C-eigenspace

Let $C = \sum_i \gamma_i e_i e_i^\mathsf{T}$ be the eigendecomposition of the structured PSD matrix of Part I (§3.3), with $\gamma_1 \geq \cdots \geq \gamma_d \geq 0$ and orthonormal C-eigenvectors e_i.

Proposition 8.7 (C-eigenspace validity)**.** *Every kernel evaluation depends on x only through the quadratic forms $x^\mathsf{T}Cx = \sum_i \gamma_i (e_i^\mathsf{T}x)^2$ and $x^\mathsf{T}Cx'$; and $\nabla_x K$ lies in the column space of C, with its component along e_i scaled by γ_i. Consequently, gradient ascent already concentrates movement along the top C-eigenvectors, and truncating to the leading C-eigen-subspace is (at full rank) an* exact reparameterization *and (at reduced rank) drops only near-zero-gradient directions.*

Proof. The kernel enters x solely via $x^\mathsf{T}Cx + \sigma_0^2 = v_x$ and $x^\mathsf{T}Cx' + \sigma_0^2$ (through $\mathcal{M}$ and $\cos\theta$); in the C-eigenbasis $x^\mathsf{T}Cx = \sum_i \gamma_i (e_i^\mathsf{T}x)^2$, so a direction e_i with $\gamma_i \approx 0$ contributes ≈ 0 to *every* kernel value. The input gradient of the arc-cosine kernel (Part I §3.5, (10)) is

$$\nabla_x K(x, x') = \tfrac{1}{\pi}\Big[(\pi - \theta)\, Cx' + \sin\theta \sqrt{v_{x'}/v_x}\; Cx\Big],$$

every term of which is $C\cdot$(vector); projecting onto e_i multiplies by γ_i. Thus movement along small-γ_i directions is intrinsically suppressed. Removing those directions (a) does not change the optimum (they carried near-zero gradient) and (b) removes flat directions, giving a better-conditioned, lower-effective-dimension problem. □

So the C-eigenspace projection is a *convergence aid / exact reparameterization*, not a constraint on the input distribution (analogous to the eigenvalue threshold on $\widetilde{K}$ in Part II §5.1).

8.3.2 Three bases and the Szegő/Fourier equivalence

All three write $x_{\text{loc}} = \bar{x} + (\text{basis}) \cdot \zeta$ and share one optimizer/diagnostics; they differ only in the basis.

- **PCA.** Columns $\Phi_K = [\phi_1, \ldots, \phi_K] =$ top-K eigenvectors of the data covariance $\Sigma_{\text{data}} = \frac{1}{N} X_c^\mathsf{T} X_c$ (eigenvalues $\nu_1 \geq \nu_2 \geq \ldots$). The offset $\bar{x}$ is *mathematically intrinsic* (centering before SVD); truncation is a *genuine data-driven constraint* (excludes low-data-variance directions). At variance threshold 0.80: $K = 197$ of the 2356 localized-support coordinates. Gradient projects as $\nabla_\zeta U_{\text{DA}} = \Phi_K^\mathsf{T} \nabla_x U_{\text{DA}}$. PCA captures *second-order statistics only* (mean + covariance); it is blind to phase structure, sparsity, and non-Gaussian marginals.

- **C-eigenspace.** Columns $E_K = [e_1, \ldots, e_K] =$ top-K C-eigenvectors. The offset is *not* intrinsic (the eigendecomposition passes through the origin) — see the caveat below. Truncation is the reparameterization of Proposition 8.7. The spectrum is extremely skewed (top eigenvalue $\sim 44\%$ of the total): at relative threshold 10^{-3}, $K = 28$ of 2356 captures 99.7% of the eigenvalue mass. (Float32 noise floor: eigenvalues below $\sim 10^{-7}\gamma_1$ are numerical noise; thresholds below $\sim 10^{-6}$ pull in garbage eigenvectors.)

- **Combined.** Project C into the PCA subspace and re-diagonalize: $C_{\text{pca}} = \Phi_K^\mathsf{T} C\, \Phi_K = Q\, \Gamma_Q Q^\mathsf{T}$, keep the columns Q above the eigen-threshold, basis $= \Phi_K Q$. Directions that are *both* data-typical (PCA) and kernel-visible (C-eigen). At $(0.80, 10^{-3})$: $2356 \to K_{\text{pca}} = 197 \to K_{\text{comb}} = 28$, achieving near-identical utility to PCA with $\sim 7\times$ fewer dimensions.

Proposition 8.8 (PCA $\approx$ Fourier under stationarity (Szegő))**.** *If the input process is second-order stationary on a regular grid, its covariance is (block-)Toeplitz, $\Sigma[i, j] = f(|i - j|)$. By Szegő's theorem, as $d \to \infty$ the eigenvectors converge to Fourier modes and the eigenvalues to the power spectral*

density $S(\omega) = \sum_k f(k)e^{-i\omega k}$. *The input distribution has* $S(\omega) \sim 1/|\omega|^2$, *so* Σ_{data}*'s eigenvectors approach the Fourier basis and PCA-truncation approximates keeping the* K *lowest-frequency Fourier modes.*

The equivalence is weakened by the irregular masked region, the non-stationarity the localization weight α injects (grid points near the center $\boldsymbol{\xi}_0$ have higher variance, breaking Toeplitz structure), finite $N \approx 10^4$ in $d \approx 900$, and residual non-stationarity of the input distribution; at $\sim 30 \times 30$ grid patches it is reasonable.

8.3.3 Subspace ≠ distributional constraint

All three are *subspace* constraints, not *distributional* ones. None guarantees x^* is a plausible sample of $p(x)$: an input with the right power spectrum but wrong phase (correlated noise) satisfies all three. A true distributional constraint needs higher-order structure (phases, sparsity) — i.e. a generative model, which is the motivation for the guided-diffusion synthesis of Part IV. The subspace approach is a useful *diagnostic* (does restricting to the right power spectrum change utility? does the GP utility operate primarily at second order?), not a full fix.

Caveat. The C-eigenspace offset is not mathematically motivated. Unlike PCA, where the mean offset is intrinsic to centering, the C-eigendecomposition passes through the origin, so setting $\bar{x} =$ mean has no principled basis — it is forced by *limiter precision*: the non-capturable eigenvectors collapse to 0 instead of to the mean, so the offset is set to the mean to compensate. This changes the utility landscape (the kernel sees $C(\bar{x} + E_K\zeta)$, not $C(E_K\zeta)$) and should be investigated. It is an acknowledged modeling wrinkle, not a resolved result.

8.4 Divergence theorems (Proof I): norm scaling and the utility blow-up

The arc-cosine kernel is positively homogeneous; scaling an input along a fixed direction inflates the posterior variance as the square of the scale while preserving the conditioning benefit, so unconstrained gradient ascent on the DA utility diverges. It is proved once here and reused by §8.5. Throughout, $x^* = \kappa\, x_t$ with scale $\kappa > 0$, and $q \equiv x_t^\mathsf{T} C x_t$.

Theorem 8.9 (Cross-kernel proportionality, $\sigma_0^2 = 0$)**.** *Let* $\sigma_0^2 = 0$ *and* $x^* = \kappa\, x_t$, $\kappa > 0$. *Then* $K(x^*, z) = \kappa\, K(x_t, z)$ *for every* z, *and hence* $k(x^*) = \kappa\, k(x_t)$.

Proof. With $\sigma_0^2 = 0$, $v_{x^*} = \kappa^2 q$ and $v_{x_t} = q$. For any z with $v_z = z^\mathsf{T} C z$,

$$\cos\theta(x^*, z) = \frac{(\kappa x_t)^\mathsf{T} C z}{\sqrt{\kappa^2 q \cdot v_z}} = \frac{\kappa\, x_t^\mathsf{T} C z}{\kappa\sqrt{q\, v_z}} = \frac{x_t^\mathsf{T} C z}{\sqrt{q\, v_z}} = \cos\theta(x_t, z),$$

so $\theta(x^*, z) = \theta(x_t, z)$ and $J(\theta(x^*, z)) = J(\theta(x_t, z))$. Then $K(x^*, z) = \frac{1}{\pi}\sqrt{\kappa^2 q\, v_z}\, J(\theta) = \kappa \cdot \frac{1}{\pi}\sqrt{q\, v_z}\, J(\theta) = \kappa\, K(x_t, z)$. Applying this to each inducing point $z = \widetilde{z}_j$ gives $k(x^*) = \kappa\, k(x_t)$. □

Corollary 8.10 (Perfect posterior correlation)**.** *Under Theorem 8.9,* $\rho_\lambda^2(x^*, x_t) = 1$.

Proof. From $k(x^*) = \kappa\, k(x_t)$ and (with $x^* \parallel x_t$, $\theta = 0$, $J(0) = \pi$) $K(x^*, x_t) = \kappa\, K(x_t, x_t) = \kappa\, v_{x_t}$, substitute into Σ_q with $W = \widetilde{K}^{-1}(V - \widetilde{K})\widetilde{K}^{-1}$:

$$\Sigma_q(x^*, x_t) = \kappa\, v_{x_t} + \kappa\, k(x_t)^\mathsf{T} W k(x_t) = \kappa\, \Sigma_q(x_t, x_t), \quad \Sigma_q(x^*, x^*) = \kappa^2\, \Sigma_q(x_t, x_t),$$

so $\rho_\lambda^2 = [\kappa\, \Sigma_q(x_t, x_t)]^2 / [\kappa^2 \Sigma_q(x_t, x_t) \cdot \Sigma_q(x_t, x_t)] = 1$. □

Corollary 8.11 (Moment scaling)**.** *Under Theorem 8.9,* $\mu(\kappa x_t) = \kappa\, \mu(x_t)$ *and* $\sigma^2(\kappa x_t) = \kappa^2\, \sigma^2(x_t)$.

Proof. Substitute $k(x^*) = \kappa\, k(x_t)$ into $\mu = k^\mathsf{T}\widetilde{K}^{-1}m$ (linear in k) and $\sigma^2 = K(x^*, x^*) + k^\mathsf{T}Wk$ (quadratic in k, with $K(x^*, x^*) = \kappa^2 v_{x_t}$). □

By Corollary 8.10, $\rho_\lambda^2 = 1 \Rightarrow \sigma^2_{\text{cond}}(x^*) = 0$: conditioning removes *all* the variance. Yet by Corollary 8.11 the marginal moments still grow ($\mu_g \propto \kappa$, $\sigma_g^2 \propto \kappa^2 \Rightarrow H_{\text{marg}} \to \infty$), so U_{DA} still diverges. In the exact parallel case ($\theta = 0, \sigma_0^2 = 0$), $\mu_{\text{cond}} = \mu$, $\sigma^2_{\text{cond}} = 0$, and H_{cond} collapses to a single Poisson entropy $H_{\text{Poisson}}(e^{\mu_g})$, $\mu_g = A\,\mu(x^*) + \lambda_0$; then (73) becomes $U_{\text{DA}}(\kappa x_t) = H_{\text{marg}}(\mu_g, \sigma_g^2) - H_{\text{Poisson}}(e^{\mu_g})$ with $\mu_g = A\kappa\,\mu(x_t) + \lambda_0$, $\sigma_g^2 = A^2\kappa^2\,\sigma^2(x_t)$.

8.4.1 Growth rate of the diverging utility

Proposition 8.12 (Gaussian observation model: $\log\kappa$ growth)**.** *If observations were Gaussian, $r(x) \sim \mathcal{N}(\lambda(x), \tau^2)$ with fixed τ^2, then for $\rho_\lambda^2 = 1$, $\sigma^2 = \kappa^2\sigma^2(x_t)$, the DA utility grows as $\log\kappa$.*

Proof. $H_{\text{marg}} = \frac{1}{2}\log(2\pi e(\sigma^2 + \tau^2))$ and, since $\sigma^2_{\text{cond}} = 0$, $H_{\text{cond}} = \frac{1}{2}\log(2\pi e\,\tau^2)$, so $U_{\text{DA}} = \frac{1}{2}\log(1 + \sigma^2/\tau^2)$. With $\sigma^2 = \kappa^2\sigma^2(x_t)$ and large κ, $U_{\text{DA}} \approx \frac{1}{2}\log(\kappa^2\sigma^2(x_t)/\tau^2) = \log\kappa + \text{const}$. □

Unbounded but slow: $dU_{\text{DA}}/d\kappa \sim 1/\kappa$, so ascent slows but never stops. Norm scaling is *not* unique to Poisson — the distinction is the *rate*.

Proposition 8.13 (Poisson-GP: κ^2 heuristic bound)**.** *For the Poisson-GP case, under the heuristic $H_{\text{marg}} \geq H_{\text{Poisson}}(\mathbb{E}[f])$ with $\mathbb{E}[f] = e^{\mu_g + \sigma_g^2/2}$ and the large-rate Poisson entropy $H_{\text{Poisson}}(\lambda) \approx \frac{1}{2}\log(2\pi e\lambda)$,*

$$H_{\text{marg}} \geq \tfrac{1}{2}\log(2\pi e) + \tfrac{1}{2}(\mu_g + \sigma_g^2/2), \qquad \textit{dominant term } \tfrac{1}{4}\sigma_g^2 = \tfrac{1}{4}A^2\kappa^2\,\sigma^2(x_t).$$

Since $H_{\text{cond}} = H_{\text{Poisson}}(e^{\mu_g})$ depends only on μ_g (grows as κ, not κ^2),

$$U_{\text{DA}} = H_{\text{marg}} - H_{\text{cond}} \;\gtrsim\; \tfrac{1}{4}\sigma_g^2 \;\propto\; \kappa^2$$

(faster than the Gaussian $\log\kappa$).

Caveat. The bound $H_{\text{marg}} \geq H_{\text{Poisson}}(\mathbb{E}[f])$ is *plausible but not proved.* A Poisson mixture over log-normal rates is overdispersed ("more spread"), but the Poisson is *not* the max-entropy law on $\mathbb{N}_0$ under a mean constraint alone (the geometric is); it is max-entropy only under Var = mean, so the naive argument does not directly apply. Numerically, fitting $U_{\text{DA}} \sim \kappa^\alpha$ over the Laplace-valid range $\kappa \in \{2, 5, 10\}$ (all $\sum p(r) > 0.95$) gives $\alpha \approx 1.9$, consistent with κ^2. The Laplace approximation truncates at $r_{\max}$ and breaks once σ_g^2 puts mass above $r_{\max}$; the safety score $z_{\text{safe}} = (\log r_{\max} - \mu_g)/\sqrt{\sigma_g^2}$ falls below 2 (unreliable) at $\kappa \approx 10$ in-model, limiting the verifiable range.

8.4.2 The $\sigma_0^2 > 0$ damping and the alignment peak at $\kappa = 1$

With $\sigma_0^2 > 0$ the results hold only approximately:

$$v_{\kappa x_t} = \kappa^2 q + \sigma_0^2 \neq \kappa^2 v_{x_t}, \qquad \cos\theta(\kappa x_t, z) = \frac{\kappa\, x_t^\mathsf{T} C z + \sigma_0^2}{\sqrt{(\kappa^2 q + \sigma_0^2)\, v_z}} \neq \cos\theta(x_t, z).$$

The fractional error $|K(\kappa x, z) - \kappa K(x, z)|/|\kappa K(x, z)|$ converges to a *constant* $O(\sigma_0^2/v_{x_t})$ as $\kappa \to \infty$ (not to zero: σ_0^2 shifts the angle by an amount $\propto \sigma_0^2/v_{x_t}$, independent of κ). In a fitted model instance ($v_{x_t} = q + \sigma_0^2 \approx 669$, $\sigma_0^2 \approx 0.98$, so $\sigma_0^2/v_{x_t} \approx 0.0015$): the measured fractional error in

Table 2: Fitted model ($M = 50$, $N = 50$; $A = 0.0265$, $\lambda_0 = -1.686$). The small gain A compresses the raw GP mean into the useful range, so the DA loophole ($U_{\mathrm{DA}} = 0.800$) is real, not a numerical artifact; the standard-optimized point overflows float32.

Input	μ	σ^2	μ_g	σ_g^2	θ(rad)	H_{marg}	U_{DA}
x_t (target)	7.86	70.9	-1.48	0.050	0	0.593	0.010
x^*_{DA}	101	3240	0.99	2.28	0.114	2.836	0.800
x^*_{Std}	68497	2.85×10^8	1816	2.0×10^5	0.866	~ 0	∞

$K(\kappa x_t, \widetilde{z}_j) \to\approx 0.6\%$ for large κ, $\rho^2_\lambda \to\approx 0.987$, and the residual variance ratio $\approx 1.3\%$ (conditioning stays very effective, not perfect). Crucially, the cosine similarity

$$\cos\theta(\kappa) = \frac{\kappa q + \sigma_0^2}{\sqrt{(\kappa^2 q + \sigma_0^2)(q + \sigma_0^2)}}$$

is *strictly maximized at* $\kappa = 1$ when $\sigma_0^2 > 0$ (and $\equiv 1$ for all κ when $\sigma_0^2 = 0$). So σ_0^2 breaks scale invariance: the kernel *wants* to match the target in direction *and* magnitude — but the magnitude still scales as $K(x^*, x^*) = \kappa^2 q + \sigma_0^2 = O(\kappa^2)$, so the bounded correlation reward cannot compete with the unbounded variance reward, and ascent still runs $\kappa \to \infty$.

Remark 8.14 (Root cause and remedy)**.** The mechanism is a decoupling of norm from direction: $K(\kappa x, y) = \kappa K(x, y)$ at $\sigma_0^2 = 0$ lets ascent raise variance (hence entropy and utility) via the norm degree of freedom without sacrificing directional alignment. Any practical gradient-based input optimization with this kernel needs a norm constraint or a normalized kernel (§8.5). The blow-up is strongly angle-dependent: effective at small angles ($\theta < 0.2$, strong conditioning), negligible at large angles ($\theta > 0.6$).

Table 2 shows the fitted-model evidence: the DA-optimized input finds a *genuine* loophole ($U_{\mathrm{DA}} = 0.800$ at a small angle $\theta = 0.114$, variance ratio $0.060 \Rightarrow \rho^2_\lambda \approx 0.94$), while the standard-optimized input diverges to $U_{\mathrm{std}} = \infty$ by numerical overflow at $\mu_g = 1816$ — two different phenomena (§8.6 separates them).

8.5 Kernel solutions (Proof II)

Two kernel-level remedies bound the variance term that §8.4 showed to be the culprit. Decompose the utility (Poisson link, $f = e^\lambda$) schematically as

$$U_{\mathrm{std}}(x^*) \;\approx\; \underbrace{\tfrac{1}{2}\sigma^2(x^*)}_{\text{variance term, } O(\kappa^2)} \;+\; \underbrace{I(\rho^2_\lambda)}_{\text{correlation term, } \leq \text{const}},$$

where the correlation term is bounded (maximized at $\kappa = 1$ once $\sigma_0^2 > 0$) but the variance term is unbounded; to fix the divergence, bound the variance term.

8.5.1 Normalized arc-cosine kernel (project onto the unit sphere)

Proposition 8.15 (Scale-invariant normalized kernel)**.** *Define* $\bar{K}(x, x') = K(x, x')/\sqrt{K(x, x)\,K(x', x')}$. *For the order-1 arc-cosine kernel,* $\bar{K}(x, x') = J(\theta)/\pi$, *and for any* $\kappa > 0$, $\bar{K}(\kappa x, \kappa x) = 1 = \bar{K}(x, x)$ *— the prior variance is strictly constant, independent of* $\|x\|$.

Proof. By Lemma 8.1 $K(x,x) = v_x$, so $\bar{K}(x,x') = [\frac{1}{\pi}\sqrt{v_x v_{x'}}\, J(\theta)]/\sqrt{v_x v_{x'}} = J(\theta)/\pi$, which depends only on θ. In particular $\bar{K}(x,x) = J(0)/\pi = 1$; and $\bar{K}(\kappa x, \kappa x) = K(\kappa x, \kappa x)/\sqrt{K(\kappa x, \kappa x)^2} = 1$. □

With $\bar{K}(x,x) = 1$ and cross terms $\bar{K}(x, \tilde{z}_j) = J(\theta_j)/\pi \in [0,1]$ bounded, the whole posterior-correction term is bounded, so $\bar{\sigma}^2(x)$ is bounded regardless of $\|x\|_C$ and U_{DA} cannot diverge by scaling: the only way up is to improve ρ_λ^2 (small angle to x_t; with $\sigma_0^2 > 0$ retained inside θ, minimized only when $x \to x_t$ in both direction and magnitude).

Caveat. The normalized kernel discards all norm information, but the input norm carries genuine predictive signal (the target depends on input magnitude, not direction alone). Empirically, training with $\bar{K}$ drops test correlation by $\sim 25\%$ — the norm encodes real structure the model needs. A fix for the input-synthesis divergence is not a free improvement to the fitted model.

8.5.2 Saturation kernel (arc-sin)

The arc-cosine kernel is the covariance of infinite-width ReLU networks, which do *not* saturate; a bounded activation gives a self-magnitude that saturates instead of growing without bound. The arc-sin (probit-activation) kernel models saturation:

$$K_{\mathrm{sat}}(x,x') = \sigma_f^2 \cdot \tfrac{2}{\pi} \arcsin\left(\frac{x^\mathsf{T} C x' + \sigma_0^2}{\sqrt{(1 + x^\mathsf{T} C x + \sigma_0^2)(1 + x'^\mathsf{T} C x' + \sigma_0^2)}} \right).$$

Proposition 8.16 (Bounded self-variance)**.** $\lim_{\|x\|\to\infty} K_{\mathrm{sat}}(x,x) = \sigma_f^2 \cdot \frac{2}{\pi} \arcsin(1) = \sigma_f^2$.

Proof. As $\|x\| \to \infty$ the ratio inside arcsin tends to $(x^\mathsf{T} C x)/(1 + x^\mathsf{T} C x) \to 1$, and $\arcsin(1) = \pi/2$, so $K_{\mathrm{sat}}(x,x) \to \sigma_f^2 \cdot (2/\pi)(\pi/2) = \sigma_f^2$. □

Unlike $K(x,x) \propto \|x\|^2$, the variance saturates to the constant σ_f^2. This fixes the divergence *without* normalizing the kernel: H_{marg} cannot grow indefinitely with norm, so to raise utility the optimizer must lower H_{cond} by finding x^* highly correlated with x_t; inputs of unbounded norm yield diminishing returns, embedding the constraint that input magnitude beyond a point yields no more signal variance.

8.6 Numerical analysis of the standard utility

This subsection is entirely about the standard utility (57), $U_{\mathrm{std}} = H_{\mathrm{marg}} - \mathbb{E}_g[H_{\mathrm{noise}}] = I(R; g \mid x^*, \mathcal{D})$. The count index r below indexes the count observation (y of Part I); the count random variable is $R \sim \mathrm{Poisson}(e^g)$.

8.6.1 The closed-form decomposition

With $\lambda(x^*) \mid \mathcal{D} \sim \mathcal{N}(\mu, \sigma^2)$, $g = A\lambda + \lambda_0$, so $g \mid \mathcal{D} \sim \mathcal{N}(\mu_g, \sigma_g^2)$, and $R \mid g \sim \mathrm{Poisson}(e^g)$:

$$H_{\mathrm{marg}} = -\sum_{r=0}^{\infty} p(r) \log p(r), \qquad p(r) = \int \mathrm{Poisson}(r; e^g)\, \mathcal{N}(g; \mu_g, \sigma_g^2)\, dg,$$

the marginal (Poisson–log-normal) entropy of (53) (no closed form; Laplace-approximated below). The subtracted expected-noise entropy *is* closed-form via the log-normal MGF ($\mathbb{E}[e^g] = e^{\mu_g + \sigma_g^2/2}$,

$\mathbb{E}[ge^g] = e^{\mu_g+\sigma_g^2/2}(\mu_g+\sigma_g^2)$):

$$\mathbb{E}_g[H_{\text{noise}}] = \underbrace{-e^{\mu_g+\sigma_g^2/2}(\mu_g+\sigma_g^2-1)}_{\text{exact (MGF), closed form}} + \underbrace{\sum_{r=0}^{\infty} p(r)\log r!}_{\text{needs } p(r)\text{, Laplace}}, \tag{76}$$

so $U_{\text{std}} \approx -\sum_{r=0}^{r_{\max}} p(r)\log p(r) - \left(-e^{\mu_g+\sigma_g^2/2}(\mu_g+\sigma_g^2-1) + \sum_{r=0}^{r_{\max}} p(r)\log r!\right)$; both sums truncate at the same $r_{\max}$.

8.6.2 Laplace approximation and the log-space Lambert-W fix

The saddle point of $p(r)$ is at the mode $\bar{g}_r$ of $\text{Poisson}(r; e^g)\,\mathcal{N}(g; \mu_g, \sigma_g^2)$, solving $r = e^{\bar{g}_r} + (\bar{g}_r - \mu_g)/\sigma_g^2$:

$$\bar{g}_r(r) = r\sigma_g^2 + \mu_g - W_0(\sigma_g^2\, e^{\,r\sigma_g^2+\mu_g}), \tag{77}$$

W_0 the principal Lambert branch. **The overflow failure mode:** forming $\sigma_g^2 e^{\,r\sigma_g^2+\mu_g}$ literally overflows float32 whenever $r\sigma_g^2+\mu_g \gtrsim 88$ ($e^{88} \sim 10^{38}$); masking the overflowing r lets the truncated $\sum p(r)$ exceed 1 by large factors. (This is the historic "$\exp(\mu)$ diverges.") **The log-space fix** never forms e^y: set $y = \log\sigma_g^2 + r\sigma_g^2 + \mu_g$ (finite for finite inputs) and solve

$$w + \log w = y \quad\Longleftrightarrow\quad w = W_0(e^y)$$

by a fixed 10 Newton iterations directly on the log-form (the backward derivative uses $dW/dy = W/(1+W)$, in which e^y cancels), then $\bar{g}_r(r) = r\sigma_g^2 + \mu_g - w$. Since $w \approx y - \log y$ for large y, $\bar{g}_r(r) \approx -\log\sigma_g^2 + \log y \sim \log r$, so $e^{\bar{g}_r(r)} \approx r$ (exactly right: the Poisson likelihood is maximized when rate = count) and stays in float32 range for any r — so $\exp(\bar{g}_r)$ is no longer an overflow risk post-fix.

8.6.3 Four independent residual failure modes

Distinct thresholds and remedies (they coincide only because all worsen at large μ_g/σ_g^2).

(a) Laplace approximation error in $\log p(r)$. A local quadratic model of the integrand; inaccurate when it departs from a quadratic-log shape. Two regimes: $\sigma_g^2 \to 0$ (prior degenerates, saddle singular — at $\sigma_g^2 < 10^{-6}$ one uses the *exact* Poisson $\log p = \mu_g r - e^{\mu_g} - \log r!$ instead); σ_g^2 large (flat prior; the asymmetric Poisson factor skews the integrand). *Independent of* $r_{\max}$ — each $p(r)$ has its own bias, observable as $\sum p(r) \neq 1$ even with no missing mass (up to 1.015; that is Laplace error, not truncation). Measured (vs numerical quadrature): error $< 1\%$ for $\sigma_g^2 \lesssim 1$ at every μ_g, growing to 5–20% at the peak by $\sigma_g^2 \sim 50$ (worst where the predictive peak slides onto $r = 0$). The driver is σ_g^2, not μ_g; float32 vs float64 shifts $\log p(r)$ by $\leq 3.7 \times 10^{-4}$ nats. *Range gate:* the count is bounded in the regime of interest ($\lesssim 100$ per observation window), so the sensible regime is $\sigma_g^2 \lesssim 6$, within which (a) is a few-percent effect.

(b) Truncation at $r_{\max}$. If the predictive puts non-negligible mass above $r_{\max}$, both $-p(r)\log p(r)$ and $p(r)\log r!$ are clipped, biasing *both* H_{marg} and the closed-sum term low. The moments are

$$\mathbb{E}[R] = e^{\mu_g+\sigma_g^2/2}, \qquad \text{Var}(R) = e^{\mu_g+\sigma_g^2/2} + e^{2\mu_g+\sigma_g^2}(e^{\sigma_g^2}-1),$$

the second (log-normal-of-rate) term dominating once $\sigma_g^2 \gtrsim 1$. As σ_g^2 grows the predictive is strongly right-skewed and its centres separate: the mean $e^{\mu_g+\sigma_g^2/2}$ climbs, the median stays $\approx e^{\mu_g}$, the mode drifts *down* toward 0 ($\approx e^{\mu_g-\sigma_g^2}$) — so "$p(r)$ peaks at e^{μ_g}" is only the small-σ_g^2 limit. A sufficient benign-truncation condition is $r_{\max} \gtrsim \mathbb{E}[R] + k\sqrt{\mathrm{Var}(R)}$. An adaptive rule is stricter (a k-σ upper tail in g-space, exponentiated to a rate, plus a Poisson std, plus a margin):

$$r_{\max}^{\text{adapt}} = e^{\mu_g+k\sigma_g} + 5\sqrt{e^{\mu_g+k\sigma_g}} + 10, \quad k = 3,$$

clamped to $[200, 10000]$, raising an error (rather than silently over-truncating) if the needed $r_{\max}$ exceeds the ceiling; a further guard triggers if the required bound exceeds e^{80}, so the overflow clamp cannot hide failures. The fixed default $r_{\max} = 100$ is benign for confident predictions but *not safe in general*: it fails whenever $\mu_g \gtrsim \log 100 \approx 4.6$ *or* σ_g^2 grows large — exactly the high-uncertainty regime active learning explores.

(c) Float32 overflow of the exact closed-form term. The exact term $-e^{\mu_g+\sigma_g^2/2}(\mu_g+\sigma_g^2-1)$ overflows float32 when $\mu_g + \frac{1}{2}\sigma_g^2 \gtrsim 88$, sending $\mathbb{E}_g[H_{\text{noise}}] \to -\infty$ and $U_{\text{std}} \to +\infty$. This is the *only* residual "exp can diverge" issue, and it lives in an *exact analytical* term, not the Laplace approximation — distinct from the mode-finding overflow above. In practice truncation (b) bites long first ($\sigma_g \sim 1 \Rightarrow$ truncation near $\mu_g \sim 1$ at $r_{\max} = 100$, vs $\mu_g \sim 88$ here).

(d) Catastrophic cancellation. Inherent to the decomposition (survives even with exact Laplace and no truncation). *Inside* $\mathbb{E}_g[H_{\text{noise}}]$ (large, earliest): both $-e^{\mu_g+\sigma_g^2/2}(\mu_g+\sigma_g^2-1)$ and $\sum p(r)\log r!$ grow $\sim e^{\mu_g}$, while the true $\mathbb{E}_g[H_{\text{noise}}]$ grows only *linearly* ($\sim \frac{1}{2}\mu_g$); at $\mu_g = 10$ each term $\sim 2\times 10^5$ vs a true value ~ 6 (~ 4.5 digits cancel), at $\mu_g = 15 \sim 6.7$ digits cancel — essentially noise in float32's ~ 7-digit mantissa (and truncating the second sum makes this *worse*). *Between* H_{marg} and $\mathbb{E}_g[H_{\text{noise}}]$ (smaller, second-level): by Jensen $H_{\text{marg}} \geq \mathbb{E}_g[H_{\text{noise}}]$ while both grow linearly in μ_g, a second cancellation costing fewer digits. Unfixable without rewriting the decomposition so cancelling pieces are computed together (e.g. summing $-p(r)\log p(r) - p(r)\log r!$ as a single term before any large analytical exponential). Flagged, out of scope.

8.6.4 "Utility diverges under gradient ascent," re-examined

The claim is *two* distinct things. (i) **Kernel-driven input blow-up:** $K(\kappa x, \kappa x) = \kappa^2 K(x,x) \Rightarrow$ ascending U_{std} with no norm constraint sends $\sigma^2 \propto \|x\|^2 \to \infty$, hence $\sigma_g^2 \to \infty$ (the mechanism of §8.4–8.5). (ii) The **computed** utility then misbehaves via the four numerical modes, typically in order: truncation collapse of H_{marg} (b), then Laplace error at large σ_g^2 (a), then cancellation inside $\mathbb{E}_g[H_{\text{noise}}]$ (d) as μ_g grows, finally float32 overflow (c) at extreme scales. The kernel makes the inputs unbounded; then numerics make the computed utility ill-behaved — not the same thing, and the four numerical effects are not the same thing either. Numerical fixes alone will not cure it: either constrain the optimizer (norm projection, or the normalized kernel of Proposition 8.15) or rewrite the closed-form term in an asymptotically stable form.

Caveat. Utility-accuracy ceiling (known limitation). The standard utility can run to $+\infty$ and the DA utility can silently collapse, both via the $r_{\max} = 100$ truncation, at high rate; a bound on the maximal rate $f_{\max}$ keeps the operating point in the accurate regime. This is a known limitation, not a resolved result.

Part IV
From selection to synthesis: guided diffusion

9 Guided diffusion for input synthesis

Parts I–III chose the next input by *selecting* the highest-utility member of a fixed pool. This part develops the natural extension: *synthesize* a new input, drawn from the input (data) distribution $p(x)$, that maximizes the same Gaussian-process (GP) utility for a given target model. A denoising diffusion model supplies a learned prior over $p(x)$, and the GP utility gradient—the very object of Part III—steers generation toward high-utility inputs. The two ingredients divide the labor cleanly: the utility gradient (smooth, low-pass; §9.1) fixes *where* in input space to move, and the diffusion score supplies the high-frequency structure of $p(x)$ that a utility gradient can never produce on its own.

9.1 Motivation: synthesize rather than select

Why direct gradient ascent on the utility fails. Suppose we optimize an input x^* directly to maximize a GP utility $U(x^*)$ by gradient ascent. The gradient flows through the arc-cosine kernel's input-gradient $\nabla_x K$ (Part I, (10)), which carries the structured (grid) prior C and hence its Gaussian smoothing block C_{smooth}—a convolution with a Gaussian kernel, i.e. a *low-pass filter*. Two consequences follow, independent of the inducing points, the training data, or which utility is used:

1. the utility gradient is *inherently smooth*, so ascent iterates lack the high-frequency structure (sharp transitions, fine texture, local contrast) that characterizes draws from $p(x)$; and
2. ascent pushes grid points outside the bounded input box. The arc-cosine self-kernel is positively homogeneous, $K(x, x) = x^\mathsf{T} C x + \sigma_0^2 = \|x\|_C^2 + \sigma_0^2$, so larger norms are rewarded and the utility diverges under input scaling (the norm-scaling divergence of Part III).

Such iterates are not admissible inputs: to lie in the support of $p(x)$ they must be indistinguishable from genuine draws from the input distribution.

Why a subspace constraint is not enough. Restricting the optimization to a low-dimensional subspace (PCA space, or the C-eigenspace of Part III) does not cure this. PCA space fixes only the *second-order* statistics; under approximate stationarity of $p(x)$ its eigenvectors tend to Fourier modes (Szegő's theorem), so PCA-space ascent is essentially Fourier low-pass filtering. Neither PCA space nor the C-eigenspace enforces the higher-order structure (phase coherence, sparse gradients) that places a sample in the support of $p(x)$: a subspace constraint is a convergence aid, not a $p(x)$-fidelity constraint.

The decomposition, and why diffusion. The synthesis task has two objectives that pull apart cleanly: (i) *informativeness*—x^* maximizes the utility (reduces output uncertainty); and (ii) *fidelity*—x^* is a plausible draw from the input distribution $p(x)$. Gradient ascent handles (i) but not (ii). A diffusion model trained on samples from $p(x)$ learns the *score* $\nabla_x \log p_t(x)$ at every noise level t; at low noise this encodes the full statistical structure of $p(x)$. Guided generation adds the two:

$$\underbrace{s_\theta(x_t, t)}_{\text{high-freq. structure}} \quad + \quad \underbrace{w\, \nabla_{x_t} U(\widehat{x}_0)}_{\text{low-freq. utility bias}} \quad . \tag{78}$$

Remark 9.1 (Division of labor)**.** The smoothness that ruined direct optimization becomes acceptable under (78): the utility only has to *bias* generation toward informative regions, not prescribe individual grid points. The score model supplies all orders of the statistics of $p(x)$ (trained once, reused across targets and utilities); the utility gradient supplies the target-specific, low-frequency direction. They act at complementary spatial-frequency scales.

Connection to the selection setting. In the fixed-pool active-learning loop, one *selects* the pool input of highest expected information gain about the latent function. Diffusion-guided synthesis is the extension: *manufacture* an input of even higher utility that still lies in the support of $p(x)$—closing the gap between "best available pool input" and "best attainable informative input." This part is framed by three questions: (1) can we generate inputs that are simultaneously plausible under $p(x)$ and high-utility? (2) how does that input change as the GP sharpens from $N = 50$ to 300 training inputs? (3) how much utility does the $p(x)$-fidelity constraint cost—the gap between the unconstrained pure-utility optimum and the diffusion-guided one, as a function of guidance strength w?

9.2 Diffusion fundamentals

9.2.1 Forward (noising) process

Let $x_0 \in \mathbb{R}^d$ be a clean input ($d = 64 \times 64 = 4096$ here). Fix a variance schedule $\beta_t \in (0, 1)$, $t = 1, \dots, T$, and define the per-step signal retention $\alpha_t = 1 - \beta_t$ and its cumulative product $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \prod_{s=1}^{t}(1 - \beta_s)$. The forward process adds Gaussian noise one step at a time,

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t;\ \sqrt{1 - \beta_t}\, x_{t-1},\ \beta_t\, I\big), \tag{79}$$

and admits a closed-form marginal at arbitrary t (the key property—no intermediate steps are needed):

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t)\, I\big). \tag{80}$$

Equivalently, by the reparameterization

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{81}$$

The schedule is chosen so that $\bar{\alpha}_T \approx 0$, hence $q(x_T \mid x_0) \approx \mathcal{N}(0, I)$: at the end of the forward process all information about x_0 is destroyed.

9.2.2 Score $\leftrightarrow$ noise, and the training objective

Differentiating the log of the forward marginal (80) gives the score in closed form, which identifies it with the added noise:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{1 - \bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1 - \bar{\alpha}_t}} \quad \Longrightarrow \quad \nabla_{x_t} \log p_t(x_t) \approx s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}. \tag{82}$$

A network trained to predict the noise therefore approximates the score. Training minimizes the simplified denoising-score-matching (DSM) L_2 loss

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_{t \sim U(1,T)}\, \mathbb{E}_{x_0 \sim p(x)}\, \mathbb{E}_{\epsilon \sim \mathcal{N}(0,I)} \Big[\big\| \epsilon_\theta(x_t, t) - \epsilon \big\|^2 \Big], \tag{83}$$

with x_t formed by (81). Each mini-batch draws a fresh (x_0, t, ϵ); the network learns all noise levels though it sees each infrequently.

9.2.3 Reverse (denoising) process and the Tweedie estimate

The generative reverse process is Gaussian, $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$, with mean read off from the noise predictor

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{1-\beta_t}}\Big[x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\,\epsilon_\theta(x_t, t)\Big], \tag{84}$$

and fixed variance σ_t^2, either the simple choice $\sigma_t^2 = \beta_t$ or the posterior form $\tilde\beta_t = \beta_t(1-\bar\alpha_{t-1})/(1-\bar\alpha_t)$; in practice $\sigma_t^2 = \beta_t$ works well. Sampling runs $t = T, \dots, 1$ from $x_T \sim \mathcal{N}(0, I)$, drawing $x_{t-1} = \mu_\theta + \sigma_t z$ with $z \sim \mathcal{N}(0, I)$ ($z = 0$ at $t = 1$). At any t the noise prediction yields the *Tweedie estimate* of the clean input—the model's posterior mean $\mathbb{E}[x_0 \mid x_t]$:

$$\widehat{x}_0 = \frac{x_t - \sqrt{1-\bar\alpha_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar\alpha_t}}. \tag{85}$$

It is coarse at high noise and increasingly detailed as $t \downarrow$. The Tweedie estimate (85) is the workhorse of guidance (§9.3): it maps a noisy intermediate to a clean input on which the GP utility can be evaluated, and it is a differentiable function of x_t.

Remark 9.2 ($T-1$ start)**.** At $t = T$ the schedules used here have $\bar\alpha_T \sim 10^{-3}$, so $1/\sqrt{\bar\alpha_T} \sim 31$ and the first reverse step amplifies any prediction error $\sim 31\times$, causing divergence. Sampling therefore starts the loop at $t = T-1$ (where $1/\sqrt{\alpha} \approx 2$, stable).

9.2.4 The U-Net noise predictor and DDIM acceleration

Because $\epsilon_\theta(x_t, t)$ maps a noisy input to a same-size tensor, it is parameterized by an encoder–decoder with *skip connections* (a U-Net): a plain bottleneck would discard exactly the fine detail that distinguishes a draw from $p(x)$ from noise, whereas skip connections carry per-resolution detail directly to the decoder. The scalar timestep t enters through a sinusoidal embedding (Transformer-style),

$$\mathrm{emb}(t)_{2k} = \sin\!\big(t/10000^{2k/d_{\mathrm{emb}}}\big), \qquad \mathrm{emb}(t)_{2k+1} = \cos\!\big(t/10000^{2k/d_{\mathrm{emb}}}\big),$$

passed through a small MLP and *added* (broadcast over space) to each block's feature map—a global "how much to activate at this noise level" modulation.

Full DDPM sampling needs $T-1 = 999$ sequential U-Net passes. *DDIM* replaces the stochastic reverse chain by a deterministic one over a subsequence of S timesteps $\tau_S > \cdots > \tau_1$ (e.g. $S = 50$). Going from $t = \tau_i$ to $t' = \tau_{i-1}$,

$$x_{t'} = \sqrt{\bar\alpha_{t'}}\,\widehat{x}_0 + \sqrt{1-\bar\alpha_{t'}-\sigma^2}\;\frac{x_t - \sqrt{\bar\alpha_t}\,\widehat{x}_0}{\sqrt{1-\bar\alpha_t}} + \sigma\, z, \tag{86}$$

with $\widehat{x}_0$ from (85). Here $\sigma = 0$ gives a fully deterministic, reproducible sampler; the DDPM choice of σ recovers stochastic sampling. Quality degrades below $S \approx 20$; $S = 50$ is the default.

9.3 Guidance: steering the reverse process with the GP utility gradient

9.3.1 Classifier-style guidance

Model "high utility" as a label y with $p(y \mid x_0) \propto \exp\big(w\,U(x_0)\big)$, $w > 0$. Bayes' rule splits the *conditional* score into a $p(x)$-fidelity term and a guidance term:

$$\nabla_{x_t} \log p(x_t \mid y) = \underbrace{s_\theta(x_t, t)}_{\text{unconditional score, (82)}} + w\,\nabla_{x_t} U_{\mathrm{std}}(\widehat{x}_0(x_t, t)). \tag{87}$$

The utility is defined on *clean* inputs, so it is evaluated on the Tweedie estimate (85) (the standard Dhariwal–Nichol trick). Any of the Part III utilities may play the role of U in (87): the standard utility U_{std} ((57)), shown here, or the distribution-aware utility U_{DA} ((50)). Define the *guidance gradient*

$$g_t = \nabla_{x_t} U_{\text{std}}(\widehat{x}_0(x_t, t)). \tag{88}$$

9.3.2 The guided reverse update (two equivalent forms)

Converting the guided score (87) back to a noise prediction, and substituting into the Tweedie map (85), yields two algebraically identical updates:

$$\epsilon_{\text{guided}} = \epsilon_\theta(x_t, t) - w\sqrt{1-\bar{\alpha}_t}\, g_t, \tag{89}$$

$$\widehat{x}_0^{\text{guided}} = \widehat{x}_0 + \frac{w\,(1-\bar{\alpha}_t)}{\sqrt{\bar{\alpha}_t}}\, g_t. \tag{90}$$

Form (89) feeds the DDPM mean (84) in place of ϵ_θ; form (90) feeds the DDIM update (86) in place of $\widehat{x}_0$. The *ramp factor* $(1-\bar{\alpha}_t)/\sqrt{\bar{\alpha}_t}$ in (90) is large at high noise ($t \approx T$, ~ 158), giving bold utility steering while global structure is still being decided, and small at low noise ($t \approx 1$, ~ 0.3), preserving fine detail.

9.3.3 The GP link: what the guidance signal is

The guidance gradient (88) is the gradient of the GP information utility of Part III, back-propagated through Tweedie. Both utilities are differentiable in the input, and the gradient flows through the arc-cosine kernel's input-gradient $\nabla_x K$ (Part I, (10)). Because $\nabla_x K$ carries the factor C—and thus the Gaussian spatial smoother C_{smooth}—the utility gradient is exactly the smooth, low-pass signal of §9.1. This is precisely why guidance works: the utility supplies a *low-frequency* informative direction, and the score model supplies the *high-frequency* structure of $p(x)$ the smooth utility gradient can never produce.

The locality mask ⇒ guidance touches only the localized-region grid points. The structured prior applies a Gaussian locality envelope to each grid point j,

$$\alpha_j = \exp\Big(-\frac{\|\boldsymbol{\xi}_j - \boldsymbol{\xi}_0\|^2}{4\,\beta^2}\Big), \tag{91}$$

where $\boldsymbol{\xi}_j$ is grid point j's coordinate, $\boldsymbol{\xi}_0$ the center of the localized region, and β the *kernel's localized-region half-width*. Grid points with $\alpha_j < 10^{-3}$ are masked out (zero contribution), so $\nabla_x U$ is nonzero *only at localized-region grid points*. The mask area scales as β^2: at the default $\beta = 0.1$ the localized region covers $\approx 21\%$ of the 64×64 grid (≈ 869 grid points), versus 5.4% at $\beta = 0.05$ and 1.7% at $\beta = 0.03$. The utility therefore guides only the localized region, and *the diffusion model freely generates every remaining grid point* for $p(x)$-fidelity—an advantage over the L_2 approach (§9.3.5), whose non-localized-region grid points are frozen at a start input. Because β is a learnable kernel parameter, it must be frozen during GP training for synthesis, or the intended mask size drifts.

Caveat. The symbol β in (91) is the *kernel* localized-region half-width (Part I), **not** the diffusion noise-schedule variance β_t of (79); likewise the locality envelope α_j is the kernel locality mask, not the diffusion signal-retention α_t. These collisions (β, α) are the most dangerous in the whole document; the macros keep them apart.

9.3.4 Full versus approximate guidance gradient

By the chain rule g_t factors through the utility gradient in clean-input space and the Tweedie Jacobian,

$$\frac{\partial \widehat{x}_0}{\partial x_t} = \frac{1}{\sqrt{\bar{\alpha}_t}}\Big(I - \sqrt{1-\bar{\alpha}_t}\,\frac{\partial \epsilon_\theta}{\partial x_t}\Big), \tag{92}$$

where $\partial\epsilon_\theta/\partial x_t$ is the $d \times d$ U-Net Jacobian, which is never formed explicitly.

- **Full gradient:** retains $\partial\epsilon_\theta/\partial x_t$ in (92); one U-Net forward *and* one backward pass, $\approx 2\times$ the cost of a plain step.
- **Approximate gradient** (default): treat the U-Net Jacobian as negligible. Then $\widehat{x}_0$ depends on x_t only through the direct $x_t/\sqrt{\bar{\alpha}_t}$ term, so

$$g_t^{\text{approx}} = \frac{1}{\sqrt{\bar{\alpha}_t}}\,\nabla_{\widehat{x}_0} U_{\text{std}}(\widehat{x}_0). \tag{93}$$

  No U-Net backward pass ($\approx \frac{1}{2}$ the cost). It is justified when ϵ_θ is a "local correction" whose Jacobian is roughly zero-mean (does not systematically align with or oppose the utility gradient), which holds best at *low* noise (small $\sqrt{1-\bar{\alpha}_t}$); a hybrid (approximate for early $t > 200$, full for late $t < 200$) captures most of the benefit cheaply.

9.3.5 The four approaches (A–D) and the adopted method

Four guidance paradigms were catalogued; all attempt to add a fidelity term counteracting the utility gradient's drift off the data manifold $\mathcal{M}$ (the support of $p(x)$). Let x_c denote a current input being optimized.

A — $\mathbf{L_2}$ penalty toward the Tweedie estimate. Corrupt x_c at a *fixed* noise level t^*, predict $\hat{\epsilon}$, form $\widehat{x}_0$ ((85)), hold it fixed (no gradient through it), and penalize the distance to it:

$$\mathcal{L}_{\text{A}} = -U_{\text{DA}}(x_c) + \frac{\gamma_{\text{reg}}}{2}\,\|x_c - \widehat{x}_0\|^2, \qquad x_c \leftarrow x_c + \eta\,\nabla U_{\text{DA}} + \eta\,\gamma_{\text{reg}}\,(\widehat{x}_0 - x_c). \tag{94}$$

Here γ_{reg} is the approach-A regularization weight. Since $x_c - \widehat{x}_0 \propto (\hat{\epsilon} - \epsilon)$, (94) is a Score-Distillation-Sampling gradient with a different time weighting. *Weaknesses:* no signal near $\mathcal{M}$ (for x_c already on $\mathcal{M}$, $\widehat{x}_0 \approx x_c$ so $\hat{\epsilon} - \epsilon$ is just reconstruction noise—the penalty is reactive, not preventive); the Tweedie estimate of an amplified input is itself amplified, so norm growth is not prevented; t^* is arbitrary; a single fixed noise sample is biased.

B — direct score. Use $s_\theta(x_t, t^*) = -\hat{\epsilon}/\sqrt{1-\bar{\alpha}_{t^*}}$ directly, chain-ruled to x_c-space. Uses $\hat{\epsilon}$ alone (not $\hat{\epsilon} - \epsilon$), so it has signal on $\mathcal{M}$ but high variance, and estimates the score of the *blurred* p_{t^*}, not p_0.

C — score without added noise. Feed x_c at $t = 1$ directly. Appealing ($p_1 \approx p_0$) but fails: a utility-distorted x_c is far out-of-distribution for a network expecting "a draw from $p(x)$ plus $\sigma \approx 0.01$ noise," and dividing by $\sqrt{1-\bar{\alpha}_1} \approx 0.01$ amplifies the unreliable prediction $\sim 100\times$.

D — full guided reverse diffusion (*adopted*). Do not optimize an existing input—*generate* from noise, nudging each denoising step by the utility gradient via (87)–(90). The U-Net is *always* in-distribution (x_t is genuinely at noise level t because it came from the reverse process); generation traverses coarse→fine over all noise levels; there is no t^* to choose; the output is a proper sample from the guided distribution. Costs: the U-Net forward (with gradients in full mode), T (or ~ 50 DDIM) sequential steps, utility evaluated on rough early Tweedie estimates, stochastic output (best-of-K), and no start-point control.

Remark 9.3 (Why the score is a genuine restoring force)**.** Naive per-coordinate clipping $\Pi_{\mathcal{B}}$ (each x_i clamped to $[x_{\min}, x_{\max}]$) acts on each grid point independently and ignores the joint distribution $p(x)$: it lands on the boundary of the input box but far from $\mathcal{M}$. In the guided score (87) the score term is instead a *structured* restoring force on the joint distribution—if a region saturates and loses its characteristic spectrum, $p_t(x_t)$ drops sharply and the score rearranges grid points to restore the correlations of $p(x)$ while accommodating the utility bias. The limitation is out-of-distribution failure: ϵ_θ was trained only on the forward trajectory (samples from $p(x)$ plus Gaussian noise), so too large a w drives x_t where ϵ_θ extrapolates and the restoring force becomes unreliable. Thus w must be large enough to explore high-utility regions yet small enough to keep x_t where the score approximation is valid.

9.3.6 The rate guard

At high noise the Tweedie estimate is coarse and can produce extreme grid points, hence extreme rates that fall in the regime where the truncated utility is inaccurate (Part III, §8.6). The generator therefore zeroes the guidance gradient whenever the predicted log-rate is too high:

$$g_t \leftarrow 0 \quad \text{whenever} \quad \exp(\mu_g) \geq f_{\max}, \qquad \mu_g = A\,\mu + \lambda_0, \tag{95}$$

with $f_{\max} = 100$ and μ_g the predicted log-rate mean (gain A times the posterior latent mean μ plus the offset λ_0). This guard activates mostly in the first ~ 10 steps (high t) and keeps generation in the low-rate regime where the (truncated) utility matches a Monte-Carlo evaluation. Because the utility landscape over random seeds is sparse (most seeds land at $U \approx 0$), generation is run best-of-K: retain the highest-utility input over K random seeds.

9.4 Open issues

Two remaining questions are mathematical rather than procedural.

Utility-accuracy ceiling at high rate. Both utilities inherit the Poisson-sum truncation at $r_{\max}$ analyzed in Part III (§8.6): above rate ≈ 50 the standard utility U_{std} overflows to $+\infty$ (its analytic noise-entropy term is unbounded), while the distribution-aware utility U_{DA} stays finite but is silently wrong (its marginal entropy collapses toward 0). Guidance is therefore reliable only in the low-rate regime, rate ≈ 0.01–1, where both utilities agree with a Monte-Carlo evaluation to within $\sim 3\%$; the rate guard (95) is exactly what confines generation to that regime. Extending accurate utility evaluation above the truncation bound—so that guidance remains valid at high rate—is open.

Single-sample distribution-aware estimator. In the guided reverse path the distribution-aware utility (50) is evaluated with a single conditioning sample ($n_{\text{mc}} = 1$, the current input) rather than a Monte-Carlo average over $p(x)$; enlarging n_{mc} is a straightforward but untested refinement.

A Notation reference

One symbol denotes one concept throughout. Entries marked **[clash]** are symbols that collide across the literature and are disambiguated here.

Symbol	Concept
Model, kernel, structured prior	
x, x^*	input (a real-valued signal on a 2-D grid); candidate input
$\boldsymbol{\xi}_i$, $\boldsymbol{\xi}_0$	grid coordinate; center $(\varepsilon_{0x}, \varepsilon_{0y})$ of the localized region
$\lambda(x)$	latent GP function [**clash: not** λ_0]
λ_0	rate offset
$f(x) = e^{A\lambda+\lambda_0}$	rate (Poisson intensity) [**clash:** f **is the rate, never the latent**]
$g = A\lambda + \lambda_0$	log-rate (so $f = e^g$)
A	gain [**clash: not the projection** a]
y $(\equiv r)$	count observation (synonyms r, n)
$K(x, x')$	arc-cosine prior kernel function
$\widetilde{K} = K(\widetilde{Z}, \widetilde{Z})$	inducing gram [**clash: never a bare** K **for this**]
$\mathcal{M} = \sqrt{v_x v_{x'}}$	kernel magnitude [**clash: not** M]
θ, $J(\theta)$	kernel angle; angular term [J **is never a Jacobian**]
$v_x = x^\mathsf{T} C x + \sigma_0^2$	kernel self-magnitude
C, C_{smooth}	structured (grid) prior covariance; RBF smoothing factor
α_i	localization weight [**clash: not diffusion** α_t]
β	half-width of the localized region [**clash: not diffusion** β_t]
ρ	smoothness SD [**clash: not correlation** ρ_λ]
σ_0	bias std, $\sigma_0 = e^{\text{raw\_sigma\_0}}$
Amp	amplitude — kernel hyperparameter, Amp = softplus$(\cdot)$, bounds $(0, 1000]$, optimized
Variational inference and eigenspace	
$\widetilde{z}$, $\widetilde{Z}$	inducing point; inducing-point matrix
$\widetilde{\lambda}$ $(\equiv u)$	inducing latent values, $q(\widetilde{\lambda}) = \mathcal{N}(m, V)$
M, N	number of inducing points; number of training inputs (M=N in the loop)
$k(x)$, $u(x) = \widetilde{K}^{-1} k(x)$	cross-covariance vector; projection vector
m, V	variational mean and covariance (inducing space)
$\mu(x)$, $\sigma^2(x)$	posterior mean and variance of the latent
$L_{\widetilde{K}}$	Cholesky factor of $\widetilde{K}$ [**clash:** $\mathcal{L}$ **is the ELBO**]
B, Λ, n_b	eigenbasis of $\widetilde{K}$; eigenvalues (diagonal); kept dimension [Λ **not** λ]
m_b, V_b, $\widetilde{K}_b$	eigenspace mean, dense covariance, diagonal inducing gram
$a = K\widetilde{K}^{-1}$	projection matrix

Symbol	Concept
$\bar{f} = e^{A\mu + \frac{1}{2}A^2\sigma^2 + \lambda_0}$	expected rate
$\mathcal{L}$	evidence lower bound [**clash: script $\mathcal{L}$, not the Cholesky factor**]
Training-step helpers	
$g_{\mathrm{E}} = A \sum_i k_i (y_i - \bar{f}_i)$	E-step gradient helper [**clash: not the log-rate g**]
$G_{\mathrm{E}} = A^2 \sum_i \bar{f}_i k_i k_i^{\mathsf{T}}$	E-step Fisher curvature (PSD)
Utility and information	
U_{std}, U_{DA}	standard and distribution-aware utility
H_{marg}, H_{noise}, H_{cond}	marginal, aleatoric-noise, and conditional entropy [**clash: never a bare H**]
$p(x)$	input (data) distribution
$\mu_g = A\mu + \lambda_0$, $\sigma_g^2 = A^2\sigma^2$	log-rate mean and variance
$\rho_\lambda(x, x^*)$	posterior latent correlation [**clash: not smoothness ρ; ρ_λ^2 = fractional variance reduction**]
c	prior conditional covariance (the "correction" term)
κ	input scaling factor, $x^* = \kappa\, x_t$ [**clash: not c**]
σ_r^2, ζ	residual/aleatoric variance; subspace coordinate [ζ **not an inducing point**]
W_0, $\bar{g}_r$, $r_{\max}$	Lambert-W (principal); Laplace mode; count truncation
σ_f^2	saturation-kernel amplitude
Diffusion	
t	diffusion timestep
x_0, x_t, $\widehat{x}_0$	clean, noisy, and Tweedie-estimated input
β_t	noise-schedule variance [**clash: not the prior width β**]
α_t, $\bar{\alpha}_t$	signal retention; cumulative retention [**clash: not the localization weight α**]
ϵ_θ, s_θ	U-Net noise predictor; score function
w, g_t	guidance scale; guidance gradient (= the utility gradient)
γ_{reg}	regularization weight (input-optimization approach)